# Holistic Framework for Multi-Drone Data Collection in IoT Systems

JONATHAN A. DILLER, Department of Computer Science, Colorado School of Mines, USA and Department of Computer and Information Science, University of Pennsylvania, USA
QI HAN, Department of Computer Science, Colorado School of Mines, USA
MATTHEW DESAULNIERS, Department of Computer Science, Colorado School of Mines, USA

Rotary-type unmanned aerial vehicles (commonly referred to as drones) are increasingly deployed as mobile data collectors for large-scale Internet-of-Things (IoT) sensor networks. However, wireless transmission rates are inherently stochastic, causing drones to adapt their trajectories during deployment. This adaptation alters flight time and energy consumption, creating a structural mismatch between deterministic offline route optimization and real-world execution under tight battery constraints. As a result, routes that appear feasible offline may become infeasible during deployment. Existing multi-drone IoT routing methods assume deterministic communication models and static energy budgets, leading to systematic infeasibility during deployment. In this article, we present a closed-loop planning framework that empirically mitigates energy exhaustion under stochastic communication. Our architecture first computes energy-constrained routes using a global offline planner that jointly optimizes drone-to-sensor assignments, visit order, and service waypoint locations. It then evaluates route feasibility using Monte Carlo simulation with an online adaptive planner that adjusts trajectories in response to real-time networking performance. Execution statistics are fed back to iteratively calibrate planning energy allocation, yielding robust routes. We further incorporate an empirically derived travel-to-transmission tradeoff that captures how optimal drone-sensor distance depends on drone speed and data volume, and exploit this tradeoff through a polynomial-time routing and convex waypoint refinement algorithm. Extensive numerical experiments and a proof-of-concept implementation on a physical drone and wireless sensor testbed demonstrate that the proposed framework significantly reduces deployment failures while maintaining low data latency compared to fixed-margin planning approaches.

CCS Concepts: • **Computer systems organization → Robotics**; • **Computing methodologies → Multi-agent planning**; • **Networks** → *Sensor networks*.

Additional Key Words and Phrases: Multi-Agent Planning, Unmanned Aerial Vehicles, Data Muling, Robotics, Sensor Networks

**ACM Reference Format:**
Jonathan A. Diller, Qi Han, and Matthew Desaulniers. 2026. Holistic Framework for Multi-Drone Data Collection in IoT Systems. *ACM Trans. Internet Things* 7, 3, Article 0 (August 2026), 29 pages. https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Wi-Fi enabled Internet of Things (IoT) sensors are used in a variety of application areas including environmental, flora and fauna, industrial, urban, military and health monitoring. Traditionally, these sensors connect to form larger networks, forwarding data to a central data center through multi-hop routing [30]. A drawback of this approach is that the nodes closest to data sinks become weak points in the network. To mitigate this problem,

Authors' Contact Information: Jonathan A. Diller, Department of Computer Science, Colorado School of Mines, Golden, Colorado, USA and Department of Computer and Information Science, University of Pennsylvania, Philadelphia, Pennsylvania, USA, diller@seas.upenn.edu; Qi Han, Department of Computer Science, Colorado School of Mines, Golden, Colorado, USA, qhan@mines.edu; Matthew Desaulniers, Department of Computer Science, Colorado School of Mines, Golden, Colorado, USA, mdesaulniers@mines.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 2577-6207/2026/8-ART0
https://doi.org/XXXXXXX.XXXXXXX

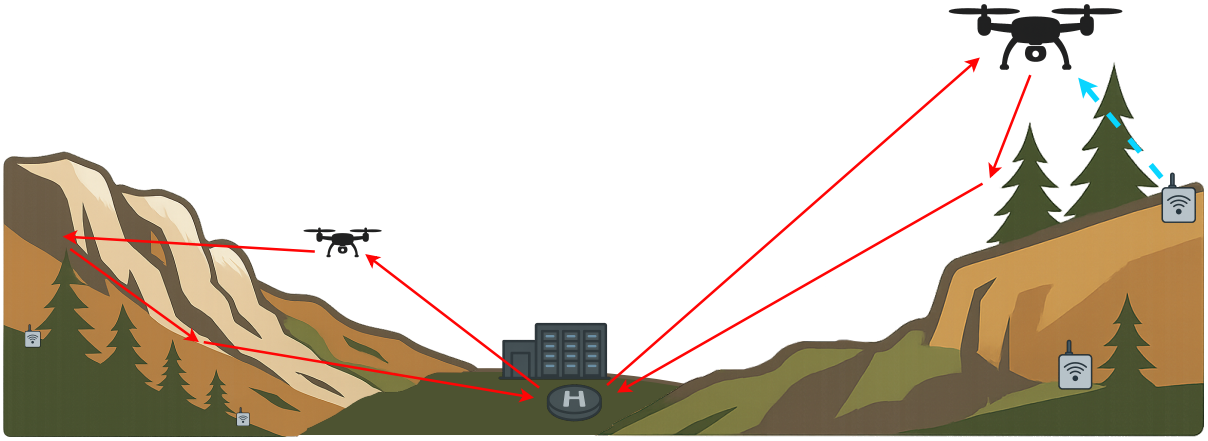


Fig. 1. Multi-drone IoT data-muling scenario.

numerous studies have explored collecting data via mobile agents, so called "data-mules" [23]. Drones are often proposed for data-muling because they can easily traverse rugged terrain, hover in place while communicating with ground nodes, and reach areas that are challenging for humans and ground robots to access. Figure 1 depicts the multi-drone data-muling scenario.

Although drones have many advantages for data-muling, there are several challenges that must be considered when planning routes for IoT data collection tasks. One major challenge in using drones is their finite on-board energy storage, requiring them to periodically recharge or change out on-board batteries. Furthermore, wireless communication can be challenging to accurately predict and failed or weak communication scenarios require the drones to adapt their behavior during deployment. This motivates the need to form efficient collection paths for multi-drone teams while accounting for limited energy and the stochastic nature of wireless communication.

Recent literature proposes numerous solutions for planning efficient data collection routes for drones. These works generally fall into one of two categories: offline mission planning [31, 46] and online adaptive behavior [11, 32]. Offline planners excel at finding near-optimal collection routes, but they tend to be overly optimistic. Because they ignore the energy costs of online adaptation, these idealistic plans often fail during deployment, risking incomplete data collection or depleted batteries. Furthermore, existing offline approaches lack data-driven feedback loops. As we observed in our previous work [8], standard offline planners and drone simulators often fail to capture the true performance and energy consumption experienced by physical hardware in the field.

*This article proposes a unifying framework to handle model mismatch between deterministic global optimization and stochastic execution in energy-constrained aerial systems for IoT data-muling.* Our framework first finds an idealistic offline plan using powerful solvers in an Offline Global Planner. The framework then refines this plan by iteratively running Monte Carlo simulations to determine realistic energy and dynamics statistics and to calibrate an energy-buffer that is fed back into the offline planner. The framework includes an Online Adaptive Planner that uses convex optimization to update waypoint locations should the drone fail to communicate with an IoT device. Additionally, our framework integrates empirical models for wireless communication and balances the tradeoff between the time it takes for a drone to travel closer to a sensor versus the time saved from the resulting increase in the data transmission rate (the so-called "travel-to-TX" tradeoff).

Building upon our preliminary work in [8], this article bridges the structural mismatch between deterministic offline planning and stochastic online execution. Specifically, this work makes the following primary contributions:

(1) **A closed-loop planning framework with deployment-aware energy calibration**: The primary novelty of this work is the closed-loop integration of deterministic offline routing, online adaptive replanning, empirical RF modeling, and Monte-Carlo feedback. Extending our prior open-loop design [8], we introduce an energy calibration loop (Section 4) that uses a stochastic drone-network simulator to empirically determine a deployment-aware energy buffer. We show empirically that this holistic integration significantly reducing the risk of infeasible plans rather than relying on idealized, fixed-margin budgets.
(2) **Empirical characterization and exploitation of the travel-to-TX tradeoff**: Moving beyond previous idealized communication models, we run field experiments to derive an empirical distance-to-throughput model (Sections 3.3 and 6.1). We use this to formally characterize the tradeoff between increasing data transfer rates and the time spent moving closer to a sensor, and we incorporate this travel-to-TX tradeoff into our proposed algorithms (Sections 5.1.1, 5.1.2, and 5.2).
(3) **A convex optimization model for refining hovering locations**: As a key enabler for both offline and online planning, we derive a second-order cone program that refines, in polynomial time, three-dimensional hovering locations for collecting IoT data.
(4) **Improved offline and online routing algorithms**: We introduce a complete algorithm to find optimal solutions to the offline path planning problem, establishing a theoretical baseline (Section 5.1.1). Furthermore, we replace the tightening heuristic from our preliminary work [8] with an improved polynomial-time algorithm that leverages convex optimization to find locally optimal offline solutions (Section 5.1.2). We apply this same optimization strategy to our online algorithm to mathematically optimize drone adaptation when communication fails (Section 5.2).

While our preliminary work established the baseline for offline data-muling, it inherently assumed an idealized, open-loop execution. The integration of the Monte Carlo feedback loop and convex waypoint refinement represents a fundamental shift from optimistic deterministic routing to closed-loop, deployment-aware planning. This transition materially changes the contribution by mathematically bridging the gap between theoretical path optimality and stochastic field execution.

To promote good research integrity and to help further the frontiers on science, we have made the source code to our framework and planning algorithms open-source[3].

The rest of this article is structured as follows: we review related works in the following section and formally introduce the Multi-Drone Data-Muling problem in Section 3. We present our holistic planning framework in Section 4, then further break down the framework into the offline global planner (Section 5.1), online adaptive planner (Section 5.2), and our combined drone and network simulator (Section 5.3). We run extensive numerical evaluations, based on field data, in Section 6 and discuss our field prototype experiments in Section 7. Finally, we discuss limitations of this work in 8, and present our conclusions and discuss future work in Section 9.

## 2 Related Work

Existing literature has addressed a variety of open problems in multi-drone data-muling. One common topic is how to cluster together sensors and select a "cluster head" [1, 15, 32, 53]. A cluster head is a single wireless node that pools together the data of neighboring nodes to transmit to the drone. A similar problem focuses on selecting hovering locations to collect data from multiple nodes at once [7, 14, 41]. A natural follow-on problem is to then determine an efficient ordering to the hovering locations [22, 26, 31, 37, 42, 46, 52]. An alternative to grouping together sensors is to have drones visit sensors individually and focus on finding efficient drone paths [4, 6, 44, 47, 51]. In this work, we do not directly consider cluster heads and focus on collecting data from nodes individually. However, existing solutions could be applied to select cluster heads that are then treated as individual nodes in our problem setup.

[3]https://github.com/JonD07/HolisticFramework

Among these problem setups, there are a variety of metrics for evaluating a solution. Some of the more common metrics include drone energy [47, 53], the total volume of data collected [3, 26], the volume of data collected from any single sensor [6, 44], the average data latency (sometimes referred to as Age-of-Information - AoI) [14], the maximum data latency (sometimes referred to as worst latency) [16, 25, 31, 52], or some combination of these metrics [41, 48]. This work uses average data latency, but our framework can be utilized with any of these metrics.

Existing approaches to these data collection problems generally fall into two categories: offline planning and online decision-making. There is a large variety of solution approaches for offline planning, many of which involve transforming the data collection problem into some other, well studied problem. One common approach for multi-drone scenarios is to divide up the sensors into groups then treat each group of sensors as a Traveling Salesman Problem (TSP) [22, 31]. Treating the problem as a variation of the TSP is also common for single drone scenarios [7]. Popular approaches for solving TSP in these works include the Genetic Algorithm (GA) [25, 32, 51], the Ant Colony algorithm [14, 39, 45], or mathematical optimization techniques [46]. Another common strategy is to treat the data collection problem as a variation of the Vehicle Routing Problem (VRP) [1, 15, 35, 37], which is solved using similar approaches as the TSP. Other related graph theory problems (and their common solutions) have also been used, such as the Orienteering Problem [26].

There are also many works that do not convert the problem into an input instance of well-studied existing solutions. Among these, formulating mathematical optimization problems is quite common [5, 6, 28, 44]. Other heuristic-based solutions include iteratively building solutions [42] or nearest-neighbor methods [4]. There are also examples of learning methods for offline single drone problems, such as [47, 48, 53] where they propose a deep Reinforcement Learning (RL) approach or [52] where they use a transformer. However, all of these offline approaches assume that the plan found offline can be executed on real hardware, ignoring the stochastic nature of wireless communication and how a drone's limited energy complicates adapting behavior online.

Online approaches predominantly leverage RL, which excels in scenarios where key aspects of the problem state are unknown *a priori*. For instance, [3, 50] tackle scenarios with unpredictable data generation rates, [49] considers unknown obstacle locations, and [24, 34] focus on highly dynamic environments. However, while RL is highly effective for navigating unknown states, they provide no guarantees of finding an optimal solution, making them an inefficient substitute for a centralized offline planner when the system state (e.g., sensor locations, data volumes) is fully known in advance. When system knowledge is available, online planning should not be tasked with solving the routing problem from scratch; rather, it should be restricted to adapting a robust offline plan. Additionally, most of these methods are designed for grid-world environments with discrete action-spaces, do not provide a mechanism for the drone-to-drone communication required to coordinate actions, and do not consider the intertwined limitations of stochastic communication and limited drone energy.

Our work leverages a centralized planner to fully exploit known system states, while strictly utilizing online planning to adapt agent behavior when unforeseen failures, such as communication drops, occur during deployment. We show how to integrate empirical wireless modeling, energy-buffer calibration via Monte Carlo validation, and convex waypoint refinement within a unified, offline-online strategy for planning and deploying multiple data-muling drones. Furthermore, the framework is modular and can easily be modified to accommodate varying problem setups.

## 3 Problem Formulation

We consider a scenario where there are $n$ heterogeneous, wireless-enabled IoT sensors deployed throughout a three-dimensional space $\mathcal{X}$ based on application needs. Each sensor is gathering application-specific data that must be collected by a drone and returned to a data center located at $x_b \in \mathcal{X}$. Let $I = \{s_1, s_2, \cdots, s_n\}$ be the set of sensors. Let the position of $s_i$ in three-dimensional space be $x_i \in \mathcal{X}$ and $q_i$ be the number of megabytes of

data to be collected from $s_i$. To prevent collisions between the drone and the physical sensor infrastructure or natural obstacles, let $d_{safe\text{-}i}$ be the minimum allowed drone altitude above sensor $i$. We assume that $x_i$, $q_i$, and $d_{safe\text{-}i}$ are all known a priori. In practical use cases (e.g., environmental sensing, agricultural monitoring), the location information of these IoT nodes and the expected data volume can be recorded during installation and are expected to be static, justifying this assumption. Let $J$ be a set of $m$ heterogeneous drones that will be launched from the data center at $x_b$ and must collect all of the captured data from the sensors and return it to $x_b$.

We formally define a drone sub-tour $\mathcal{T}$ as a sequence of tuples $\langle(\tilde{x}_i, s_i) \mid i \in \mathcal{T}\rangle$, where $\tilde{x}_i \in \mathcal{X}$ and $s_i$ is a sensor that the drone will collect data from while at $\tilde{x}_i$. It is assumed that the drone will move from $x_b$ to the first $\tilde{x}_i$ in $\mathcal{T}$ and from the final $\tilde{x}_i$ in $\mathcal{T}$ back to the data center. For operational safety and mathematical simplicity, we assume that $x_b$ is located at a predefined distance above the ground at the data center. We argue that this assumption is justified because ascending and descending between the ground and $x_b$ at the beginning and end of $\mathcal{T}$ will take a constant amount of time and energy and is independent of $\mathcal{T}$.

## 3.1 Communication Model

Mathematical models, such as the Shannon-Hartley theorem for data throughput and Friis transmission equation for radio signal strength in the Fraunhofer region, provide a theoretical model for wireless communication. However, these theorems represent the theoretical upper bound on performance in ideal conditions. Various factors, such as multi-path propagation from signals reflected from the ground, environmental noise, atmospheric conditions, other wireless devices, and the time of day, will impact actual performance [21, 43]. Furthermore, the data throughput will be bound by hardware limitations at short distances. Because of these factors, it is recommended to use empirical methods to derive models from experimental data [38].

From our field experiments, we found that the data transfer rate (in megabytes per seconds) from the sensor to the drone can be modeled as

$$tx(d_{ji}) = min\Big(\frac{a_i}{d_{ji}^2 + c_i} + b_i,\ r_{max\text{-}i}\Big), \tag{1}$$

where $d_{ji}$ is the distance from drone $j$ to sensor $i$, values $a_i, b_i$ and $c_i$ are constants and $r_{max\text{-}i}$ is the maximum expected data transfer rate. These latter four values must be determined experimentally. Equation (1) mimics the logarithmic decay expected from the Shannon-Hartley theorem at longer distances while providing an inflection point at the transition to the near-field region and a hardware limit at very close distances. We assume that there is a known minimum data transfer rate, $r_{min\text{-}i}$, that must be met to successfully transmit data between two devices. Let $d_{max}$ be the distance that achieves $r_{min\text{-}i}$ (that is, $tx(d_{max}) = r_{min\text{-}i}$).

We define *servicing* a sensor as a drone connecting wirelessly with a sensor and collecting all data held on that sensor. Suppose that a drone is hovering at $\tilde{x}_i$ when it services $s_i$ (which is located at $x_i$). The time required to service $i$ will be

$$t_h(\tilde{x}_i, x_i) = \frac{q_i}{tx(||\tilde{x}_i - x_i||_2)}, \tag{2}$$

where $||x||_2$ is the Euclidean norm of $x$ and $q_i$ is the volume of data to collect from $i$. Given a drone sub-tour $\mathcal{T}_j$, the total time spent servicing the sensors on $\mathcal{T}_j$ will be

$$t_s(\mathcal{T}_j) = \sum_{(\tilde{x}_i, s_i) \in \mathcal{T}_j} t_h(\tilde{x}_i, x_i). \tag{3}$$

We assume that only a single sensor will communicate with the drone at a time but acknowledge that in practice packet collision can be handled by the MAC layer protocol.

### 3.2 Energy Model

Drones have limited on-board energy, usually in the form of batteries, and can only operate for so long before needing to either swap out batteries or recharge. Previous works have shown that drone power consumption is a function of drone speed [29, 46], which was verified by [40]. Let $\mathcal{P}(v)$ be the function that maps speed $v$ to power consumed, in watts. If a drone travels for $t$ seconds at speed $v$ (in meters per second), the drone will consume $\mathcal{P}(v)t$ joules. Let $\beta_j$ be the total usable joules on drone $j$.

Assuming that drone $j$ moves at an average speed of $v_j$ on sub-tour $\mathcal{T}_j$, the total time required to travel $\mathcal{T}_j$ (excluding time spent hovering while servicing sensors) will be

$$t_m(\mathcal{T}_j) = \frac{1}{v_j}\big(||x_b - \tilde{x}_1||_2 + \sum_{i \in \mathcal{T}_j} ||\tilde{x}_i - \tilde{x}_{i+1}||_2 + ||\tilde{x}_\varphi - x_b||_2\big), \tag{4}$$

where $|\mathcal{T}_j| = \varphi$. The total time to complete sub-tour $\mathcal{T}_j$ will be the sum of the time to travel through the sub-tour and the time to service all sensors on the sub-tour. Mathematically,

$$t_{tot}(\mathcal{T}_j) = t_s(\mathcal{T}_j) + t_m(\mathcal{T}_j). \tag{5}$$

Similarly, let the total energy to complete $\mathcal{T}_j$ be

$$\mathcal{J}(\mathcal{T}_j) = \mathcal{P}(0)t_s(\mathcal{T}_j) + \mathcal{P}(v_j)t_m(\mathcal{T}_j). \tag{6}$$

Each drone $j$ will have a sequence of sub-tours, $\mathcal{K}_j$, that contains one or more sub-tours, $\mathcal{T}_j$. Between consecutive sub-tours, the drone must have its battery swapped out at the data center. Let $t_b$ be the time required to swap out batteries. The completion time of sub-tour $\mathcal{T}_j$ will be

$$t_{cmp}(\mathcal{T}_j) = \begin{cases} t_{tot}(\mathcal{T}_j), & j = 1 \\ t_{tot}(\mathcal{T}_j) + t_{cmp}(\mathcal{T}_{j-1}) + t_b, & j \geq 2. \end{cases} \tag{7}$$

Note that we assume that no drone will need a battery swap before the first tour, but this assumption can easily be reversed.

### 3.3 Travel-To-TX Tradeoff

Observe that the time required to service a node (Equ. 2) depends on the drone-to-sensor distance, $d_{ji}$, and the quantity of data on the sensor, $q_i$. Reducing the distance will increase the data transmission rate (Equ. 1), which reduces the service time. However, moving close to a sensor takes time - which depends on the drone's speed.

Without loss of generality, assume that there is a single sensor $s_i$ that is $x_i$ meters from the data center. If we assume that the drones do not communicate while traveling, a drone must travel $x_i - d_{ji}$ meters, at $v_j$ meters-per-second, before collecting $q_i$ MB of data to service $s_i$ and then travel $x_i - d_{ji}$ meters back to the data center. We want to select $d_{ji}$ such that we minimize the time to move $x_i - d_{ji}$ meters, collect data, and return $x_i - d_{ji}$ meters to the data center. The total time to move and collect data will be

$$t_{tot}(d_{ji}) = \frac{2(x_i - d_{ji})}{v_j} + \frac{q_i(d_{ji}^2 + c_i)}{a_i + b_i(d_{ji}^2 + c_i)}. \tag{8}$$

Note that the second term is the result of simplifying the fraction $\frac{q_i}{tx(d_{ji})}$. If we take the partial derivative of Equ. (8), we get

$$\frac{\partial t_{tot}}{\partial d_{ji}} = \frac{-2}{v_j} + \frac{2q_i d_{ji}}{a_i + b_i(d_{ji}^2 + c_i)} - \frac{2b_i q_i d_{ji}(d_{ji}^2 + c_i)}{(a_i + b_i(d_{ji}^2 + c_i))^2}. \tag{9}$$

Solving for the roots of Equ. (9) will give us the locally minimum values for $t_{tot}(d_{ji})$, one of which will be the optimal value for $d_{ji}$ to minimize the total time required to service $s_i$. Apart from the sensor-specific parameters

$(a_i,\ b_i,\ c_i)$, observe that the optimal value of $d_{ji}$ depends on $v_j$ and $q_i$. In general, we expect $d_{ji}$ to decrease when $q_i$ increases and $d_{ji}$ to increase when $v_j$ decreases. That is, it is better to get closer to collect larger volumes of data and it is better to stop early when the drone moves slowly. We refer to this as the *travel-to-TX* tradeoff.

### 3.4 Problem Definition

We can now formally define the Multi-Drone Data-Muling problem (MDDMP). Our objective is to find sub-tours that service every sensor such that we minimize the average latency of each sensor while adhering to each drone's limited energy budget. Stated mathematically: given a set of $n$ sensors, data center location $x_b$, and a team of $m$ drones, find drone sub-tours such that

$$\min \frac{1}{n} \sum_{j=1}^{m} \sum_{\mathcal{T}_j \in \mathcal{K}_j} |\mathcal{T}_j| t_{cmp}(\mathcal{T}_j), \tag{10}$$

where every sensor $s_i$ is in at least one $\mathcal{K}_j$, and

$$\mathcal{J}(\mathcal{T}_j) \leq \beta_j,\ \forall \mathcal{T}_j \in \mathcal{K}_j, j \in J \tag{11}$$

$$d_{safe\text{-}i} \leq ||\tilde{x}_i - x_i||_2 \leq d_{max}, \forall \tilde{x}_i \in \mathcal{T}_j, j \in J. \tag{12}$$

## 4 Holistic Drone Planning Framework

There is often a disparity between centralized, high-level offline planning and real-world execution caused by the simplifying assumptions made to render the problem easier to solve offline. This is particularly true when dealing with wireless devices due to the stochastic nature of radio frequencies. A variety of factors can impact a drone's ability to collect data from a sensor including mis-predicting the communication range of the sensor or a weakened sensor battery. This motivates the need to consider both offline, high-level planning and online adaptive behavior jointly.

To address the disparity in planning, we present a holistic drone planning framework for solving the MDDMP that combines offline mission planning with online adaptive behavior. Figure 2 shows an overview of our proposed framework.

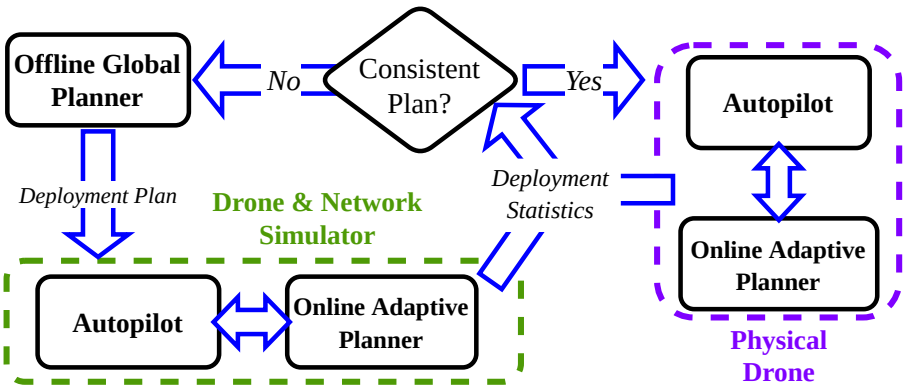



Fig. 2. Overview of our drone planning framework.

The framework first finds an efficient solution to the MDDMP problem via a Global Planner. The Global Planner assumes that drones must stop and hover to service a sensor and that each drone will move at a fixed speed.

Additionally, the Global Planner modifies Constraint (11) to be

$$\mathcal{J}(\mathcal{T}_j) \leq \alpha\beta_j, \ \forall \mathcal{T}_j \in \mathcal{K}_j, j \in A, \tag{13}$$

where $\alpha \in (0, 1]$ is the battery share alloted for offline planning. The motivation for adjusting the energy budget of each drone is to leave excess energy for adaptive behavior during deployment. Setting $\alpha$ too high will cause the drone to not have enough energy to adapt online while setting $\alpha$ too low will restrict the Global Planner and lead to less efficient plans. Ideally, $\alpha$ should be as large as possible without causing a drone to run out of energy during deployment.

The Global Planner sends its deployment plan to a drone and network simulator. The drone portion of the simulator handles realistic drone dynamics while the network portion of the simulator handles drone-to-sensor communication and simulates stochastic communication performance. In the simulator, an autopilot carries out the deployment plan and invokes the Online Adaptive Planner should the drone fail to communicate with a sensor. In the case of failure, the online planner moves the drone as close to the sensor as allowed and plans a new route back to the data center. Unlike the Global Planner, the autopilot does not force the drone to stop and hover to service a node and will attempt to communicate with node $i + 1$ as soon as it has finished servicing node $i$.

The framework simulates the plan multiple times in a Monte-Carlo fashion, collecting average drone speeds and total energy used. After several iterations of the simulator, the framework considers the performance of the deployment plan. The plan is considered *consistent* if the total energy used on each sub-tour $j$ is below but within a desirable range of $\beta_j$ and the predicted average drone speeds $v_j$ is within a desirable range of the measured $v_j$. If the deployment plan is not consistent, the framework adjusts the values of $\alpha$ and the average drone speeds, then sends this information back to the Global Planner to restart the cycle. If a deployment plan is consistent, then it is ready to be deployed on the actual drone to collect data.

It is important to clarify that this framework does not utilize Monte-Carlo simulations to generate routing models or to solve the underlying optimization problem. The combinatorial routing and waypoint selection are handled strictly by the deterministic Offline Global Planner. Instead, the Monte-Carlo simulations serve purely as a statistical stress-testing environment. If the Monte-Carlo stress tests reveal that a plan is likely to fail (i.e., deplete the battery), the framework simply adjusts the scalar energy-buffer margin ($\alpha$) and average speed parameters, passing them back to the Offline Global Planner to prompt a re-calculation of a more conservative route.

The physical drone has the same autopilot and Online Adaptive Planner as the simulator with the exception that the online planner in the simulator does not observe any drone energy constraint while the physical drone does. This is because we want the simulation to determine the actual energy required to service all of the sensors but we do not want the physical drone to crash during deployment.

To calibrate $\alpha$, we initially set it to 1 then run the Drone and Network Simulator several times to determine if the drones will run out of energy. If the drones do run out of energy, then we adjust $\alpha$. Let the maximum observed joules consumed per drone sub-tour on the $i^{th}$ iteration of the framework be $\tilde{j}_i$. If $\tilde{j}_i > \beta$ (that is, the drones are running out of energy during deployment), then we use the ratio between $\beta$ and $\tilde{j}$ as an update rule. Stated mathematically, we adjust $\alpha$ using

$$\alpha_{i+1} = \alpha_i \frac{\beta}{\tilde{j}_i}. \tag{14}$$

Lemma 1. *Given that $\alpha_1 = 1$ and $\tilde{j}_i > \beta > 0$, the sequence defined by the update rule* (14) *is strictly positive and monotonically decreasing. That is, $0 < \alpha_{i+1} < \alpha_i$ for all $i \geq 1$.*

Proof. We will use induction to prove Lemma 1. Assume that $\alpha_i > 0$ for some $i \geq 1$. Because $\tilde{j}_i > \beta > 0$, we know that

$$0 < \frac{\beta}{\tilde{j}_i} < 1. \tag{15}$$

Given that $\alpha_{i+1} = \alpha_i \phi_i$, for some $\phi_i \in (0, 1)$, it follows by induction that if $\alpha_i > 0$, then $0 < \alpha_{i+1} < \alpha_i$ and update rule (14) is strictly positive and monotonically decreasing. □

While update rule (14) formally ensures the monotonic decrease and positivity of $\alpha$, we note that convergence to an optimal feasible value under highly generalized, unbounded stochastic conditions is not mathematically guaranteed. Additionally, the Monte Carlo calibration loop provides empirical risk reduction rather than a formal feasibility guarantee. However, as we demonstrate empirically in Section 6, the loop successfully calibrates to a feasible value across typical deployment conditions.

Our planning framework has a modular design that allows individual pieces to easily be swapped. In the following section we provide details on how we chose to implement the major components of the framework.

## 5 Implementation Details

This section provides the implementation details of our proposed framework. Due to the modular design of the framework, these components can be modified or replaced without requiring the framework itself to be updated. In the following two subsections we further discuss how to implement a Global Planner and an Online Adaptive Planner. We describe the Drone and Network Simulator used for this study in Subsections 6.

### 5.1 Offline Global Planner

In this section, we present two methods for solving MDDMP. We first formulate a Mixed Integer Non-Linear Program that finds an optimal solution. This formulation provides a baseline to compare against but is not tractable for larger problem inputs. We then present a polynomial time algorithm that divides the problem into multiple Traveling Salesman Problems (TSP), that are solved using well studied heuristics-based algorithms, and then improves the performance of the solution using convex optimization. All sets used in this section are provided in Table 1 and all decision variables are listed in Table 2.

Table 1. Sets for Mixed Integer Non-Linear Program

| Set | Magnitude | Description |
|---|---|---|
| $I$ | $n$ | Set of sensors, indexed with $i$ |
| $J$ | $m$ | Set of drones, indexed with $j$ |
| $K$ | $\leq n$ | Set of drone sub-tours, indexed with $k$ |

*5.1.1 Mixed Integer Non-Linear Solution.* To solve the MDDMP to optimality, we propose a Mixed Integer Non-Linear Program (MINLP). The MINLP formulation is motivated by similar formulations for solving the Vehicle Routing Problem (VRP) and uses the well-known Miller-Tucker-Zemlin cycle elimination constraint to prevent sub-cycles in drone sub-tours. One important way that our formulation deviates from the classic VRP is that we are selecting the location of each waypoint where a drone will service each sensor.

We define continuous variables $\mathbf{x}_i, \mathbf{y}_i$, and $\mathbf{z}_i$ as the waypoint coordinates from which a drone will service node $i$. Let $K$ be the set of sub-tours that a drone has the option to fly on. Let binary variable $\mathbf{e}_{ji\hat{\imath}k}$ be a visit flag where if $\mathbf{e}_{ji\hat{\imath}k} = 1$ then drone $j$ will travel from waypoint $i$ to waypoint $\hat{\imath}$ on sub-tour $k$, and 0 otherwise. Likewise, binary variable $\mathbf{e}_{jbik}$ determines if drone $j$ travels from the data center to waypoint $i$ on sub-tour $k$ and $\mathbf{e}_{jibk}$ determines if $j$ goes from waypoint $i$ to the data center. We use binary variable $\mathbf{w}_{jk}$ to determine if drone $j$ will run sub-tour $k$. Continuous variables $\mathbf{t}_{jk}$ is the time duration of drone $j$'s $k^{th}$ sub-tour, $\mathbf{d}_{i\hat{\imath}}$ is distance from waypoint $i$ to waypoint $\hat{\imath}$, $\mathbf{d}_{bi}$ is the distance from the data center to waypoint $i$, and ${}^{n}\mathbf{d}_i$ is the distance from waypoint $i$ to sensor $i$. Integer variable $\mathbf{u}_i$ assigns a visit number to each waypoint. Continuous variables ${}^{s}\mathbf{t}_i$

Table 2. Decision Variables for Mixed Integer Non-Linear Program

| Variable | Range | Description |
|---|---|---|
| $\mathbf{x}_i, \mathbf{y}_i, \mathbf{z}_i$ | $\mathbb{R}$ | Coordinates of servicing waypoint for node $i$ |
| $\mathbf{d}_{i\hat{\imath}}$ | $\mathbb{R}$ | Distance from waypoint $i$ to waypoint $\hat{\imath}$ |
| $\mathbf{d}_{bi}$ | $\mathbb{R}$ | Distance from the data center to waypoint $i$ |
| ${}^{n}\mathbf{d}_i$ | $\mathbb{R}$ | Distance from servicing waypoint $i$ to node $i$ |
| ${}^{o}\mathbf{d}_i$ | $\mathbb{R}$ | Distance to servicing waypoint $i$ from servicing waypoint $i-1$ in $\mathcal{T}_j$ |
| ${}^{f}\mathbf{d}_b$ | $\mathbb{R}$ | Distance from final servicing waypoint in $\mathcal{T}_j$ to the data center |
| $\mathbf{t}_{jk}$ | $\mathbb{R}_{\geq 0}$ | Time duration of drone $j$'s $k^{th}$ sub-tour |
| ${}^{s}\mathbf{t}_i$ | $\mathbb{R}_{\geq 0}$ | Time required to service node $i$ |
| $\mathbf{r}_i$ | $\mathbb{R}_{\geq 0}$ | Data transfer rate when servicing node $i$ |
| $\mathbf{e}_{ji\hat{\imath}k}$ | $\mathbb{B}$ | 1 if $j$ travels from $i$ to $\hat{\imath}$ on sub-tour $k$, 0 otherwise |
| $\mathbf{e}_{jbik}$ | $\mathbb{B}$ | 1 if $j$ travels from the base to $i$ on sub-tour $k$, 0 otherwise |
| $\mathbf{e}_{jibk}$ | $\mathbb{B}$ | 1 if $j$ travels from $i$ to the base on sub-tour $k$, 0 otherwise |
| $\mathbf{w}_{jk}$ | $\mathbb{B}$ | 1 if $j$ deploys on sub-tour $k$, 0 otherwise |
| $\mathbf{u}_i$ | $\mathbb{N}$ | Sequence number for node $i$ |
| $\mathbf{n}_{jk}$ | $\mathbb{N}$ | Number of nodes drone $j$ visits on sub-tour $k$ |

track the time required to service node $i$ and $\mathbf{r}_i$ determines the data transfer rate while servicing node $i$. Integer variable $\mathbf{n}_{jk}$ for the number of nodes that drone $j$ visits on sub-tour $k$.

Our objective is to minimize the sum of individual sub-tour times:

$$\min \ \frac{1}{n}\Bigg(\sum_{j\in J}\sum_{k\in K}\Big(\mathbf{n}_{jk}\sum_{\hat{k}\in K,\hat{k}\leq k}\mathbf{t}_{j\hat{k}}\Big)\Bigg) \tag{16}$$

*subject to*

$$\frac{1}{v_j}\Big(\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{d}_{i\hat{\imath}}\mathbf{e}_{ji\hat{\imath}k}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jbik}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jibk}\Big)+\sum_{i\in I}\sum_{\hat{\imath}\in I}{}^{s}\mathbf{t}_i\mathbf{e}_{ji\hat{\imath}k}+t_b\mathbf{w}_{jk}\leq\mathbf{t}_{jk},\quad \forall j\in J,\ k\in K \tag{17a}$$

$$\mathbf{n}_{jk}=\sum_{\hat{\imath}\in I}\sum_{k\in K}\mathbf{e}_{ji\hat{\imath}k}+\sum_{i\in I}\mathbf{e}_{jbik},\quad j\in J,\ k\in K \tag{17b}$$

$$\frac{\mathcal{P}(v_j)}{v_j}\Big(\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{d}_{i\hat{\imath}}\mathbf{e}_{ji\hat{\imath}k}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jbik}+\sum_{i\in I}\mathbf{d}_{bi}\mathbf{e}_{jibk}\Big)+\mathcal{P}(0)\sum_{i\in I}\sum_{\hat{\imath}\in I}{}^{s}\mathbf{t}_i\mathbf{e}_{ji\hat{\imath}k}\leq\alpha\beta_j,\quad \forall j\in J,\ k\in K \tag{17c}$$

$$\mathcal{M}\mathbf{w}_{jk}\geq\sum_{i\in I}\sum_{\hat{\imath}\in I}\mathbf{e}_{ji\hat{\imath}k},\quad \forall j\in J,\ k\in K \tag{17d}$$

$$\mathbf{w}_{jk-1}\geq\mathbf{w}_{jk},\quad \forall j\in J,\ k\in K, k>1 \tag{17e}$$

$$\sum_{i\in I}\mathbf{e}_{jbik}=\mathbf{w}_{jk},\quad \forall j\in J,\ k\in K \tag{17f}$$

$$\sum_{i\in I}\mathbf{e}_{jibk}=\mathbf{w}_{jk},\quad \forall j\in J,\ k\in K \tag{17g}$$

$$\sum_{j\in J}\sum_{\hat{\imath}\in I}\sum_{k\in K}\mathbf{e}_{j\hat{\imath}ik}+\sum_{j\in J}\sum_{k\in K}\mathbf{e}_{jbik}=1,\quad \forall i\in I \tag{17h}$$

$$\sum_{j\in J}\sum_{\hat{\imath}\in I}\sum_{k\in K}\mathbf{e}_{j\hat{\imath}ik}+\sum_{j\in J}\sum_{k\in K}\mathbf{e}_{jibk}=1,\quad \forall i\in I \tag{17i}$$

$$\sum_{\hat{\imath}\in I}\mathbf{e}_{j\hat{\imath}ik}+\mathbf{e}_{jbik}-\Big(\sum_{\hat{\imath}\in I}\mathbf{e}_{j\hat{\imath}ik}+\mathbf{e}_{jibk}\Big)=0,\quad \forall j\in J,\ i\in I,\ k\in K \tag{17j}$$

$$\mathcal{M}(1-\mathbf{e}_{j\hat{\imath}ik})+\mathbf{u}_i+1\geq\mathbf{u}_{\hat{\imath}}\geq\mathbf{u}_i+1-\mathcal{M}(1-\mathbf{e}_{j\hat{\imath}ik}),\quad \forall j\in J,\ i,\hat{\imath}\in I,\ k\in K \tag{17k}$$

$$\mathbf{d}_{i\hat{\imath}}^2\geq(\mathbf{x}_i-\mathbf{x}_{\hat{\imath}})^2+(\mathbf{y}_i-\mathbf{y}_{\hat{\imath}})^2+(\mathbf{z}_i-\mathbf{z}_{\hat{\imath}})^2,\quad \forall i,\hat{\imath}\in I \tag{17l}$$

$$\mathbf{d}_{bi}^2\geq(\mathbf{x}_i-\pi_1(x_b))^2+(\mathbf{y}_i-\pi_2(x_b))^2+(\mathbf{z}_i-\pi_3(x_b))^2,\quad \forall i\in I \tag{17m}$$

$${}^n\mathbf{d}_i^2\geq(\mathbf{x}_i-\pi_1(x_i))^2+(\mathbf{y}_i-\pi_2(x_i))^2+(\mathbf{z}_i-\pi_3(x_i))^2,\quad \forall i\in I \tag{17n}$$

$$\mathbf{z}_i\geq d_{safe-i},\quad \forall i\in I \tag{17o}$$

$$\mathbf{r}_i\leq\frac{a_i}{{}^n\mathbf{d}_i^2+c_i}+b_i,\forall i\in I \tag{17p}$$

$$r_{max\text{-}i}\geq\mathbf{r}_i\geq r_{min\text{-}i},\forall i\in I \tag{17q}$$

$${}^s\mathbf{t}_i\geq\frac{q_i}{\mathbf{r}_i},\forall i\in I \tag{17r}$$

Constraint (17a) bounds the total time of sub-tour $k$ for drone $j$. Constraint (17c) enforces the drones energy budget on each sub-tour, where $\mathcal{P}(v)$ maps speed $v$ to power consumed, in watts, $\beta_j$ is the capacity of the drone's battery, and $\alpha$ is the battery share for offline planning. Constraint (17d) determines if a particular sub-tour $k$ is actually used for drone $j$ while constraint (17e) forces lower numbered sub-tours to be used first. Constraints (17f) and (17g) force each sub-tour that is used to start and end at the data center. Constraints (17h) and (17i) ensure that each waypoint is visited exactly once while constraint (17j) ensures that the "in" edge and "out" edge of each waypoint is on the same sub-tour for the same drone. Constraint (17k) enforces a numbering system on each waypoint to eliminate smaller cycles in drone sub-tours. Constraint (17l) tracks the distance between each waypoint, constraint (17m) tracks the distance from each waypoint back to the data center, and constraint (17n) tracks the distance from waypoint $i$ to sensor $i$. Constraint (17o) prevents the drone from striking the ground or other obstacles close to sensor $i$. Note that $\pi_i(x)$ is the projection function that returns the $i^{th}$ values of vector $x$. Together, constraints (17p) and (17q) restrict the data transmission rate for sensor $i$ based on (1), and constraint (17r) limits sensor service time based on the data transmission rate.

It is important to note the relationship between the objective formulated in Equ. (10) and the MINLP defined in Equ. (16). While (10) provides the generalized, high-level definition of the MDDMP objective (minimizing average latency subject to energy capacity), (16) serves as its direct translation into an explicit, computationally solvable format required by mathematical optimization solvers.

Although solving our MINLP will yield an optimal solution, MINLP are particularly hard to solve to optimality. Various techniques, such as column generation algorithms [33] or data-driven approaches [27], may improve the time required to find a solution to (16). However, for the purposes of our proposed framework, we would prefer a solution that is guaranteed to terminate in polynomial time. In the following subsection we present a more tractable approach to solving the MDDMP.

*5.1.2 Polynomial Time Solution.* Our MINLP has two major factors that make it hard to solve. The first is the combinatorial problem, where we must select which drones visit which sensors and in what order. The second major challenge is Constraints (17p) and (17q), which limit the transmission rate using Equ. (1) - a non-convex exponential decay function.

For a more tractable solution to the MDDMP, we propose a divide and conquer approach that overcomes the major challenges of the MINLP solution. Our algorithm treats the problem as a VRP instance and uses a heuristics-based solver to find an initial solution, which solves the combinatorial problem. After finding an initial solution, our algorithm formulates a convex optimization problem for selecting drone hovering locations to service nodes by using a straight-line approximation of Equ. (1), making the optimization problem solvable in polynomial time. We term this approach *VRP+CO*.

**Algorithm 1** Multi-Drone Data-Muling Algorithm

**Input:** $I$ : set of sensor, $x_b$ : data center position
**Output:** $\mathcal{K}$ : set of drone tours

1: $k \leftarrow 1$
2: **while** *stopping condition not met* **do**
3: $C \leftarrow k\text{-}means(I,\ x_b,\ k)$
4: **for** $C_j \in C$ **do**
5: $\mathcal{T}_j \leftarrow solve\text{-}TSP(C_j, x_b)$
6: $\mathcal{T}_j \leftarrow improve\text{-}tour(\mathcal{T}_j)$
7: *append* $\mathcal{T}_j$ *on* $\mathcal{T}$
8: **end for**
9: $\mathcal{K} \leftarrow assign\text{-}drones(\mathcal{T})$
10: **if** *each* $\mathcal{T}_i \in \mathcal{T}$ *is under energy budget* **then**
11: *Stopping condition met*
12: **return** $\mathcal{K}$
13: **else**
14: $k \leftarrow k + 1$
15: **end if**
16: **end while**

Algorithm 1 lists the details of our algorithm. We start by fixing $k$, the total number of sub-tours per drone, on line 1. We then use Lloyd's $k$-means clustering algorithm on line 3 to form $k$ clusters. On line 5 we solve a TSP on nodes in each cluster along with the data center. We use the Lin-Kernighan-Helsgaun (LKH) heuristic [20] to solve each TSP instance, which runs in polynomial time [19]. We then run the *improve-tour*() function on the newly found sub-tour, which is further described below. On line 9 we assign drone to sub-tours by sorting the drone based on total energy budget available and sorting each sub-tour by total distance, then assigning drones to sub-tours based on this ordering. On line 10 we verify that each drone will have enough energy to perform each sub-tour assigned to it. If true, then we have met the stopping condition and the algorithm ends. Otherwise, we increase $k$ and restart the algorithm.

To improve each sub-tour, the *improve-tour*() function attempts to find better hovering locations $\tilde{x}_i$ for each sensor $i$ in sub-tour $\mathcal{T}_j$. We do this by solving a convex optimization problem that simplifies the MINLP presented in Section 5.1.1. In addition to the variables present previously, let ${}^o\mathbf{d}_i$ be the distance from the hovering location before $i$ to hovering location $i$ in $\mathcal{T}_j$ and let ${}^f\mathbf{d}_b$ be the distance from the last hovering location in $\mathcal{T}_j$ back to the data center. We can formulate a convex optimization problem that minimizes time to collect data from the sensors on $\mathcal{T}_j$ as follows:

$$\min\ \frac{1}{v_j}\Big(\sum_{i\in\mathcal{T}_j} {}^o\mathbf{d}_i + {}^f\mathbf{d}_b\Big) + \sum_{i\in\mathcal{T}_j}^{s} T_i \tag{18}$$

*subject to:* Constraints (17n), (17r), (17a), (17q), and

$$ {}^{o}\mathbf{d}_i^2 \geq (\mathbf{x}_i - \mathbf{x}_{i-1})^2 + (\mathbf{y}_i - \mathbf{y}_{i-1})^2 + (\mathbf{z}_i - \mathbf{z}_{i-1})^2, \quad \forall i \in \mathcal{T}_j \tag{19} $$

$$ {}^{f}\mathbf{d}_b^2 \geq (\mathbf{x}_i - \pi_1(x_b))^2 + (\mathbf{y}_i - \pi_2(x_b))^2 + (\mathbf{z}_i - \pi_3(x_b))^2, \quad i = last_element(\mathcal{T}_j) \tag{20} $$

$$ \mathbf{r}_i \leq \frac{r_2 - r_{max\text{-}i}}{d_2 - d_1}({}^{o}\mathbf{d}_i - d_1) + r_{max\text{-}i}, \forall i \in \mathcal{T}_j \tag{21} $$

Constraint (19) tracks the waypoint-to-waypoint distance while constraint (20) tracks the distance from the last waypoint in $\mathcal{T}_j$ to the data center. Constraint (21) limits the data transfer rate for each sensor in $\mathcal{T}_j$ by linearly approximating the curve in Equation (1), where

$$ d_1 = \sqrt{\frac{a_i}{r_{max\text{-}i} - b_i} - c_i} \tag{22} $$

$$ r_2 = \frac{a_i}{d_2^2 + c_i} + b_i \tag{23} $$

and $d_2 > d_1$.

We would like to exploit the *travel-to-TX* tradeoff by solving for roots in Equ. (9) and using the optimal drone-to-sensor distance for $d_2$. However, observe that Equ. (9), when simplified, is a fifth-order polynomial, which is not solvable in radicals per the Abel-Ruffini theorem. To estimate an optimal value for $d_2$ we could use numerical methods, such as Newton's method, or by creating a look-up table with precomputed values. We chose to use a look-up table for our implementation of the *VRP+CO* algorithm to reduce computation time in our Online Adaptive Planner (as discussed in Section 5.2), but either method can be used.

The *VRP+CO* algorithm can be solved in polynomial-time. Observe that objective function (18) is linear and constraints (17n), (17r), (17a), (17q), (19), (20) and (21) are all convex, creating a second-order cone program (SOCP). While the time to solve a SOCP will depend on the solver employed, such as Gurobi – a proprietary tool without a published theoretical runtime analysis – the problems can be solved to optimality in polynomial-time using interior-point methods [36]. Furthermore, solving the VRP will be dominated by the underlying TSP solver. Solving the TSP is notoriously NP-hard. However, heuristic-based approaches like the LKH solver can find solutions in polynomial time, albeit without optimality guarantees. Gurobi and other numerical optimization tools for solving SOCPs tend to be more efficient than the LKH solver, the latter having a time complexity of $O(n^{2.2})$ [19]. The outer while-loop in our algorithm will run at most $n$ times, making the total runtime of our Multi-Drone Data-Muling Algorithm $O(n^{3.2})$. However, in the average case we do not anticipate the outer while-loop to run $n$ times, giving us a runtime between $O(n^{2.2})$ and $O(n^{3.2})$.

## 5.2 Online Adaptive Planner

Our planning framework requires a method for quickly adapting the plan found using the Offline Global Planner. For this, we propose an Online Adaptive Planner that utilizes the SOCP presented in Section 5.1.2. When communication failure occurs, the autopilot from our framework in Section 4 gives the online planner the drone's current position, the sensor that failed to respond to the drone, and a list of sensors from that sub-tour that have not yet been visited.

Suppose that sensor $\hat{\imath}$ has failed to communicate with the drone (which is located at $\tilde{x}_{\hat{\imath}}$) and the remaining sensors to visit are in $\mathcal{T}_{\hat{\jmath}}$. We formulate a new SOCP by adding the distance from $\tilde{x}_{\hat{\imath}}$ to the point at the safety altitude directly above sensor $\hat{\imath}$ into objective function (18) and to energy constraint (17c). We then solve the rest of the SOCP from Section 4 using the points in $\mathcal{T}_{\hat{\jmath}}$, giving us a new data collection plan that attempts to connect from the failing sensor while adapting the original deployment plan from the offline planner but avoids the combinatorial aspect of the MDDMP.

## 5.3 Drone & Network Simulator

Our framework requires us to run Monte Carlo simulations to validate drone deployment plans. To create a realistic environment for validating plans, we joined together a real-time drone simulator, a drone autopilot that supports adaptive behavior, and a simplified networking simulation. We discuss each of these pieces in-depth below. Although we chose specific tools for each of these elements, any of these individual pieces can be swapped out with alternative options and our framework would still be applied the same.

We used the ArduPilot Software-In-The-Loop (SITL) simulator[4] as our real-time drone simulator. The Ardupilot SITL models drone dynamics and can be set to run in any real-world location. The simulator utilizes MAVProxy's mavproxy_map module for simulating location-specific terrain features, meaning that the drone in the simulation can crash into the ground if the terrain is not accounted for (we will address this concern in the following subsection). The ArduPilot SITL supports a variety of control commands through the MAVLink protocol[5], and mirrors common commands used in commercially available flight controllers, such as the CubePilot Cube Orange Flight Controller[6], meaning that the same commands used to control a real drone can be used to control a simulated drone in the SITL simulator. This feature allows our autopilot to work on both the ArduPilot SITL simulator and on our physical drone.

*5.3.1 Drone Autopilot.* For an autopilot, we have adapted the DroNS3 autopilot described in [17]. This autopilot can read in a list of commands as a plan text file and will execute each command in the given order. At a high level, these commands fall into a handful of basic operations: takeoff, move to a waypoint, collect data from a sensor, return to the home position, and land.

We have adapted this autopilot framework in a few ways. The first adaptation is that we have combined the move-to-waypoint and the collect-data commands so that the drone will attempt to collect data while moving. We have also added an adaptive behavior feature, where the autopilot will call the Online Adaptive Planner when drone-to-sensor communication fails. Another notable adaptation is ground-avoidance. When the autopilot executes a move-to-waypoint command, we check the trajectory of the drone to verify that the drone will not collide with the ground. If a ground collision is predicted, then we replace that move-to-waypoint command with a series of shorter move commands that direct the drone to gain altitude and avoid the ground.

*5.3.2 Network Simulator.* To simulate drone-to-sensor communication, we have implemented our own simplified network simulator (termed SimpleNetSim or SNS) that utilizes a data-driven approach to simulate peer-to-peer communication. At a high level, SNS simulates a single peer-to-peer transaction by taking in the quantity of data to transmit and the distance between the two devices, then predicts the data transfer rate and simulates the time delay to transmit the given volume of data. Although SNS can be used with any volume of data, we recommend breaking up large volumes of data into smaller chunks to accommodate the drone's movement and changing drone-to-sensor distance.

SimpleNetSim uses a Weibull distribution to predict peer-to-peer data transfer rates. We chose a Weibull distribution because it was previously shown to nicely model network traffic [2] and our own field test data reaffirmed these findings. Given peer-to-peer distance $d$, the Weibull distribution's probability density function is defined as

$$f(d;\lambda,c) = \frac{c}{\lambda}\left(\frac{d}{\lambda}\right)^{c-1} e^{-(d/\lambda)^c}, \tag{24}$$

where $\lambda$ defines the shape of the distribution and $c$ defines the scale, both of which depend on distance $d$. That is, a given distance $d$ should have unique values for $\lambda$ and $c$. Let $f_W(d)$ be a function that maps distance to values for $\lambda$ and $c$. SimpleNetSim tunes function $f_W(d)$ using transmission rate measurements at varying distances

[4]https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html
[5]https://mavlink.io/en/
[6]https://docs.px4.io/main/en/flight_controller/cubepilot_cube_orange.html

from field experiments on physical hardware. Once function $f_W(d)$ has been tuned, SNS randomly draws a transmission rate for distance $d$ by drawing random variable $R_{wb} \backsim Weibull(f_W(d))$. Note that $\lambda$ and $c$ are both device dependent, meaning that function $f_W(d)$ must be tuned for each unique sensor type.

We acknowledge that we previously used NS3, a powerful discrete event networking simulator[7], in our prior work [8]. However, we found several limitations in using NS3 that SNS attempts to overcome. The first limitation is that NS3 does not readily provide stochastic behavior. SimpleNetSim overcomes this by sampling the Weibull distribution each time the simulator is called, which emulates the inconsistent behavior commonly seen during deployment. The second limitation that we found with NS3 is that it is not easy to accurately replicate the performance seen on our own hardware. SimpleNetSim is data-driven from field test data, making it easy to model our own hardware in the simulation.

## 6 Performance Evaluations

In this section we evaluate our planning framework through various numerical simulations with field data. We first discuss how we ran field tests to collect data transmission measurements and used the results to find various parameters used in our framework. We then evaluate the performance of our proposed framework and various algorithms for both the Offline Global Planner and the Online Adaptive Planner.



Fig. 3. Drone and Raspberry Pis used in field experiments.

### 6.1 Data Transmission Experiments

To determine values for Equ. (1), $\lambda$, and $c$, we deployed two Raspberry Pis (one RPi3B+ and one RPi4) in a field and flew a ROARQuad drone [18] equipped with an onboard RPi4 above them at varying distances. All devices ran headless Ubuntu 22.04. Figure 3 shows the drone and the ground nodes. At each distance, the ground nodes transmitted data to the drone over a 2.4 GHz ad-hoc Wi-Fi network to measure the TCP data transfer rate.

All devices utilized the integrated Broadcom/Cypress CYW43455 Wi-Fi chipset. The physical layer was bound to a single spatial stream (1x1 SISO) over a 20 MHz channel bandwidth. Although initialized in ad-hoc mode, the hardware automatically negotiated the 802.11n (Wi-Fi 4) standard. Rate adaptation was governed natively via the internal firmware's multi-rate retry mechanism through the FullMac driver framework, which cycled dynamically

[7]https://www.nsnam.org/

through standard 802.11n modulation and coding schemes (MCS 0-7) rather than relying on a host-side routing protocol.

Data transfer throughput was measured using a custom client-server application written in C. The application utilized a TCP connection at the transport layer to account for realistic network overhead and congestion control. The sockets were configured to be IP-version agnostic, with the server passively binding to the local interface over statically assigned IP addresses.

Figure 4 shows the results of our experiments with Equ. (1) fitted to the data. We used the random sample consensus algorithm to fit the model to our data because this approach was shown to work well with very noisy data [13]. For the RPi3 (Fig. 4a), we found $a = 681,\ b = -0.246,\ c = 110,\ r_{max} = 5.37$ and $r_{min} = 0.2$, and for the RPi4 (Fig. 4b), we found $a = 2403,\ b = -0.654,\ c = 308,\ r_{max} = 4.9$ and $r_{min} = 0.4$. We used SciPy[8] to determine $\lambda$ and $c$ at each of the various distances to build function $f_W(d)$. The values for $\lambda$ and $c$ listed in our open source repository.

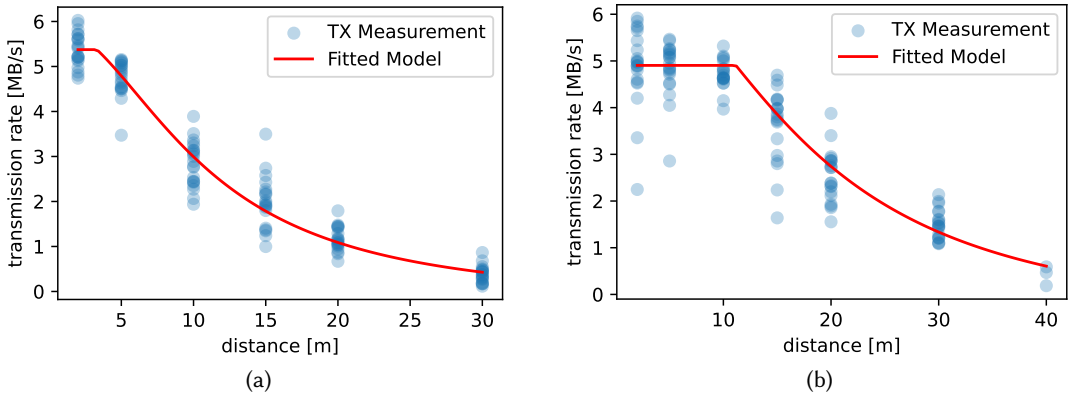



Fig. 4. Results from field experiments to determine data transfer rates between a drone using a Raspberry Pi 4 and a Raspberry Pi 3 (*a*) and Pi 4 (*b*) sensors on the ground.

Using the parameters for Equ. (1), we computed optimal values for $d_*$ in Equ. (9), for both Raspberry Pis (RPi3 in Fig. 5a and RPi4 in Fig. 5b), to build a look-up table for Constraint (21). We solved the equation with varying values for $v$, from 2 meters-per-second up to 20 meters-per-second in increments of 1. At each unique speed setting, we used values of $q = 0.001 * 2^{n/8}$ MB, for $n \in \{0, 1, \cdots 159\}$, giving us a total of 160 values for $d_*$ at each speed setting. The results of these computations is shown in Fig. 5. These graphs clearly show the *travel-to-TX* tradeoff and demonstrate that characteristics of the tradeoff are dependent on the speed of the drone and the volume of data on the sensor. The flat section at the bottom of each graph occurs when $\frac{a}{d^2+c} + b = r_{max}$, at which point there is no longer a gain in data transmission rate for moving closer to the sensor and corresponds with the flat segment of $tx(d)$ on the left side of the graphs in Fig. 4. The red dashed line shows the distance at which $\frac{a}{d^2+c} + b = r_{min}$ and is the max drone-to-sensor distance ($d_{max}$) that we expect to be able to communicate at.

## 6.2 Framework Evaluation

To evaluate our proposed framework, we generated several randomized IoT sensor deployments and evaluated how well the framework could bridge the gap between offline, global planners and adaptive online behavior. We

[8]https://docs.scipy.org/doc/scipy/

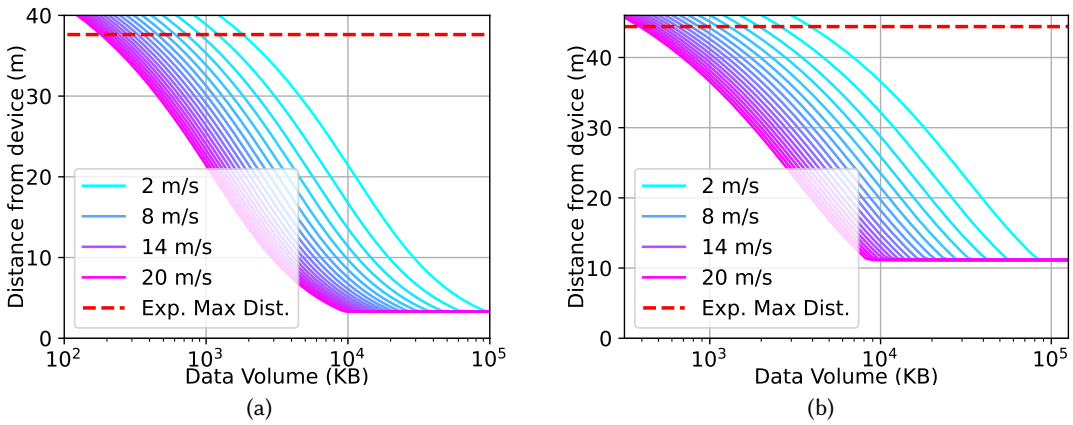



Fig. 5. Optimal distance to collect data from Raspberry Pi 3 (*a*) and Pi 4 (*b*) sensors based on drone speed.

first look at how the framework compares against using a fixed value for $\alpha$, as is common practice in multi-drone planning works that observe explicit energy constraints but do not calibrate their energy budget [10, 15, 31, 42, 44]. This approach is equivalent to running our framework (same Offline Global Planner, Online Adaptive Planner, and simulation setup) but without calibrating $\alpha$ via feedback from Monte Carlo simulations. We consider $\alpha$ fixed at 1.0, 0.75, and 0.5. We further expand this evaluation by considering two deployment strategies: (1) executing the global plan with an energy failsafe that drives the drone back home when it runs low on energy and (2) executing a planned mission without an energy failsafe. We employ the Return-to-Home failsafe feature because this is common practice in UAV operations [12], and consider the no-failsafe scenario to assess energy failure rates when blindly following a global plan. We also look at how our framework performs when increasing the number of drones, and conduct a sensitivity study on the number of Monte Carlo simulations and additional RF noise.

For these experiments, we consider a variety of metrics. We first consider average latency, as defined in Section 3.4, and the worst latency, which is defined as the time lapsed from the start of the mission until the last drone returns to the data center. Although the global planner is not optimizing worst latency, we include this metric as it is popular in multi-drone data collection applications [16, 25, 31, 52] and because our framework is independent of the metric used by the global planner. We also consider failure rate, which is defined as the number of missions executed that exceed energy budget $\beta$ over the total number of considered mission executions.

For these experiments, we generated a data set where the number of sensors varies from five up to 30, in increments of 5. We created 10 unique sensor deployments at each increment by selecting random locations within a $577 \times 577$ meter space, with the data center located at the center, which gives us a sensor density of 15 $sensors/km^2$ up to 90 $sensors/km^2$. We used MAVProxy's mavproxy_map module to determine the altitude ($z$ value) of each of the randomly placed sensors. Unless otherwise stated, we ran five Monte-Carlo iterations in our framework for each input. We note that several of the below experiments were conducted using a single-drone scenario. However, we argue that a single-drone evaluation faithfully represents the expected behavior of larger multi-drone deployments because the Online Adaptive Planner operates locally on individual vehicles and the offline planner scales by assigning disjoint sub-tours, assuming collision avoidance and multi-drone interference are handled by lower-level autonomy.

*6.2.1 Varying Number of Sensors With Failsafe.* We compared our framework against the fixed-budget approach when operating with an online energy failsafe. The failsafe triggers when the drone has spent 90% of its usable joules ($\beta$). When the failsafe triggers, the drone immediately returns back to the base to have its battery swapped. In the case of the failsafe, the same global planner is triggered again on the remaining sensors that were skipped, including sensors assigned to that drone in future sorties not yet executed. The drone then relaunches and performs the new set of plans generated by the offline global planner.

Figure 6a shows our simulation results. The top graph shows the average found average latency compared to the best-found solution and the bottom graph shows the average worst latency. We see little difference between the methods with fewer sensors ($n \leq 15$) but see an improvement in performance as the number of sensors grows, with the framework improving performance by 11.5%, 8.5%, and 7.8% when compared against $\alpha = 0.5$, 0.75, and 1.0, respectively, when ($n \geq 20$). We see a similar trend in worst latency, where our framework improves performance by 12.41%, 11.30%, and 8.28% when $\alpha$ is fixed at 0.5, 0.75, and 1.0, respectively, with ($n \geq 20$).

*6.2.2 Varying Number of Sensors Without Failsafe.* We compared our proposed framework against the fixed-budget approach when operating without an online energy failsafe. The simulation was set to flag when a drone would have run out of energy while completing an assigned flight plan, but does not stop the vehicle from completing the plan. Figure 6b shows the results of our simulations. For clarity, our adaptive approach (the purple line) is beneath $\alpha = 1$ (black line) in the top plot and beneath $\alpha = 0.5$ (red line) in the bottom graph. The top graph shows the average found average latency compared to the best-found solution. We see a fair amount of variation when the number of sensors is small ($\leq 10$), which is expected due to the stochastic nature of the networking simulator. However, as the number of sensors grows, our adaptive framework and $\alpha$ fixed at 1.0 are consistently performing the best, while fixing $\alpha = 0.5$ has high variance and averages a 15.8% increase in average latency

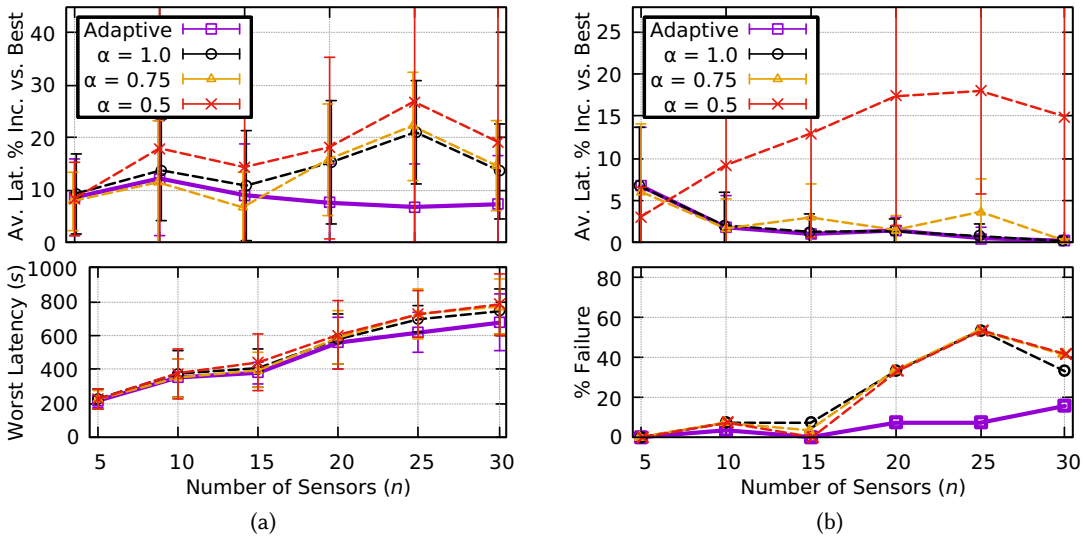



Fig. 6. Simulation results showing the impact of the number of sensors on framework performance. Plot (a) shows the percent increase in average latency (top) and average worst latency (bottom) when the drones use a return-to-base failsafe. Plot (b) shows the percent increase in average latency compared to the best-found solution (top) and the percentage of inputs that each approach failed to find a valid plan for (bottom) when the drones have no return-to-base failsafe.

when there are 15 or more sensors. This is because fixing $\alpha$ to a lower value causes the offline planner to be too conservative and generate more sub-tours than needed, leading to higher data latency.

The bottom graph in Figure 6b shows the percentage of inputs where the approach failed to find a valid solution (where the drone would have ran out of energy). When there are only 5 sensors, all approaches find valid solutions. However, as the number of sensors grows, the fixed-budget approaches fail to find an energy-safe solution on 22.3%, 23.1%, and 22.6% of all inputs with $\alpha = 0.5$, 0.75, and 1, respectively. In contrast, our adaptive approach only fails 5.6% of inputs.

*6.2.3 Varying Number of Drones.* To evaluate how the number of drones impacts performance, we ran our framework and the fixed-budget approaches on inputs with 25 sensors ($n = 25$) and increased the number of drones ($m$) from one to five. We disabled the energy failsafe and measured the percent increase in average latency compared to the best-found solution and failure rate. Figure 7a shows our results. The framework provides good solution quality while reducing failures with a high sensor-to-drone ratio. For example, adapting $\alpha$ improved solution quality by around 18% compared to the fixed approach with $\alpha = 0.5$, while simultaneously reducing the failure ratio from 53.6% to 7.2%, when the sensor-to-drone ratio is 25-to-1 (one drone). The improvement over the fixed-budget approaches starts to disappear as the sensor-to-drone ratio reaches 5-to-1 (five drones).

*6.2.4 Varying Number of Monte-Carlo Simulations.* We further evaluated the framework's sensitivity to the number of Monte-Carlo simulations ran while learning $\alpha$. For this experiment we used a single drone and fixed the number of sensors ($n$) at 20, then increased the number of Monte-Carlo simulations from one up to five. When the framework could run all Monte-Carlo simulations without finding a failure, we used the found value for $\alpha$ in the drone and networking simulator five times, measuring the number of failures. This gave us a total of 50 runs for each setting. The top plot of Fig. 7b shows our results. Although the failure rate observed in the graph is

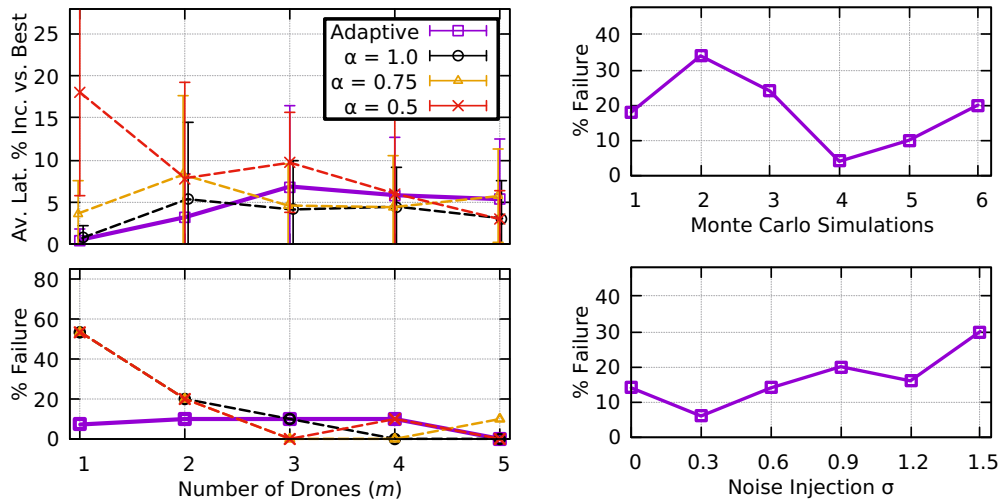



Fig. 7. Plot (a) shows the percent increase in average latency compared to the best-found solution (top) and the percentage of inputs that each approach failed to find a valid plan for (bottom) as the number of drones increases. Plot (b) shows the impact that the number of Monte-Carlo simulations (top) and additional communication noise injection (bottom) have on the framework's failure rate.

noisy, we see that it trends downwards with the number of Monte-Carlo simulations, with an average failure rate of 9.3% when the number of simulations is set to five.

*6.2.5 Varying Additional Channel Noise.* To determine the impact of communication uncertainty, we injected Additive White Gaussian Noise (AWGN) on the transmission rate in SNS to simulate additional external interference from other devices broadcasting on the same RF channel as the IoT sensors. The effective rate in SNS is sampled from

$$R_{eff} = R_{wb} - |N_{AWGN}|, \tag{25}$$

where $R_{wb}$ is the initial rate drawn from a Weibull distribution, as discussed in Section 5.3.2, and $N_{AWGN} \sim \mathcal{N}(0, \sigma^2)$ is the injected AWGN. $R_{eff}$ is then evaluated against the minimum viable rate ($r_{min\text{-}i}$) to determine if the simulated transmission was successful. For clarity, a value of $\sigma = 0$ means that there is no injected AWGN but there is still stochastic communication, as defined in Section 5.3.2. We evaluated how increasing $\sigma$ impacted the framework's failure rate. We again fixed the number of sensors ($n = 20$) and used a single drone for these experiments.

The bottom plot of Fig. 7b shows our results. The framework begins to see a notable increase in the failure rate as $\sigma \geq 0.9$. However, at smaller values of $\sigma$ the framework is still able to adapt to the additional noise and performs the similar to no injected AWGN.

*6.2.6 Framework Evaluation Summary.* These results show the need for our holistic approach to planning. Allowing an Offline Global Planner to be too optimistic (setting a higher value for $\alpha$) gives us better quality solutions but can generate infeasible plans, where the drones will run out of energy when adapting online. Conversely, allowing the Offline Global Planner to be too conservative (setting a lower value for $\alpha$) avoids failures but leads to poor quality solutions. These findings are most notable when the sensor-to-drone ratio is higher. Interestingly, the fixed-$\alpha$ approaches were still able to occasionally find a valid solution that matched the best-found solution on larger inputs (30 sensors). This tells us that there is no "perfect" value for $\alpha$ and it needs to be experimentally determined.

We also found that additional Monte-Carlo simulations help reduce the failure rate experienced by the framework. We expect that further increasing the number of Monte-Carlo simulations would reduce the failure rate when dealing with additional communication noise.

## 6.3 Offline Global Planner Evaluation

To evaluate the performance of our proposed Offline Global Planner algorithm (*VRP+CO*), we compared our solution against two baseline approaches: (1) the VRP algorithm in listing 1 without the convex optimization step (*VRP*), and (2) the "tightening heuristic" method that minimizes travel distance (*VRP+TH*). The *VRP* algorithm is simply flying the drones as close as possible to the sensor and ignoring the *travel-to-TX* tradeoff, as done in [4, 35, 39, 48], which assumes that maximizing the transmission rate always supersedes the impact of travel time. Conversely, the *VRP+TH* assumes a constant transmission rate when within some given distance to a sensor and minimizes travel time. This is similar to [1, 8, 25, 31, 51] and a variation of other works that focus on choosing a hovering location to service multiple nodes at once, such as [7, 32, 42]. For the *VRP+TH* approach, we set a transmission range of 30 meters for the RPi3 and 35 meters for the RPi4. As an additional benchmark, we implemented our MINLP introduced in Section 5.1.1. We used Gurobi, version 12.0, for the MINLP and our *improve-tour*() function in the *VRP+CO* algorithm. We used the LKH solver, version 3.0, to solve the VRP in all three heuristics-based approaches.

While there have been advances in other methods, such as RL-based approaches [3, 11, 24, 34, 49, 50], we do not compare against these methods due to the various misalignments in problem setup pointed out in Section 2. Principle among these are the lack of tightly coupled stochastic communication and energy constraints,

requiring us to fundamentally alter reward structures and state spaces, which risks unfair implementation bias. Therefore, we compare against the MINLP (theoretical optimum), VRP (minimizing communication distance), and VRP+TH (minimizing travel distance), as they isolate the specific impact of the travel-to-TX tradeoff and convex optimization under identical system limitations.

We generated three sets of randomized sensor deployments for this evaluation. The first set has a varying number of sensors from four to 30 in increments of 1 (50 layouts at each increment) where the sensors are either Raspberry Pi 3s (RPi3) or Raspberry Pi 4s (RPi4) with an average sensor density of 75 $sensors/km^2$. The sensors have varying volumes of data to collect, defined as $q = 0.001 * 2^n$ MBs for some randomly picked $n \in \{0, 1, \cdots 15\}$. We term this the *n4-30* input set. We again used MAVProxy to determine the altitude of each sensor. The second sensor deployment set has the same parameters as the *n4-30* set, but with five up to 200 sensors, in increments of 5. The second set is termed *n5-200*. Our third sensor deployment set has 50 sensors with a uniform volume of data to collect from each sensor. We set the volume of data to $q = 0.001 * 2^n$, with $n$ incrementing from zero up to 16, and generated 50 sensor deployments at each increment. We term this set *q0-16*.

We consider three types of drones in our numerical evaluation. The first drone is the model discussed in [29]. We estimate that this aircraft normally operates at 15 meters per second and carries 112 kilojoule of usable energy. The second considered drone is the one discussed in [40], which likely operates at 12 meters per second. We estimate that this drone could carry a 10.0 ampere-hour, 15.2 Volt battery, which would give the aircraft approximately 438 kilojoule of usable energy. The final considered drone is one described in [9], which operates at 10 meters per second and an estimated 360 kilojoule of usable energy. The parameters needed for $\mathcal{P}(v)$ for each drone are provided in each of the referenced publications.

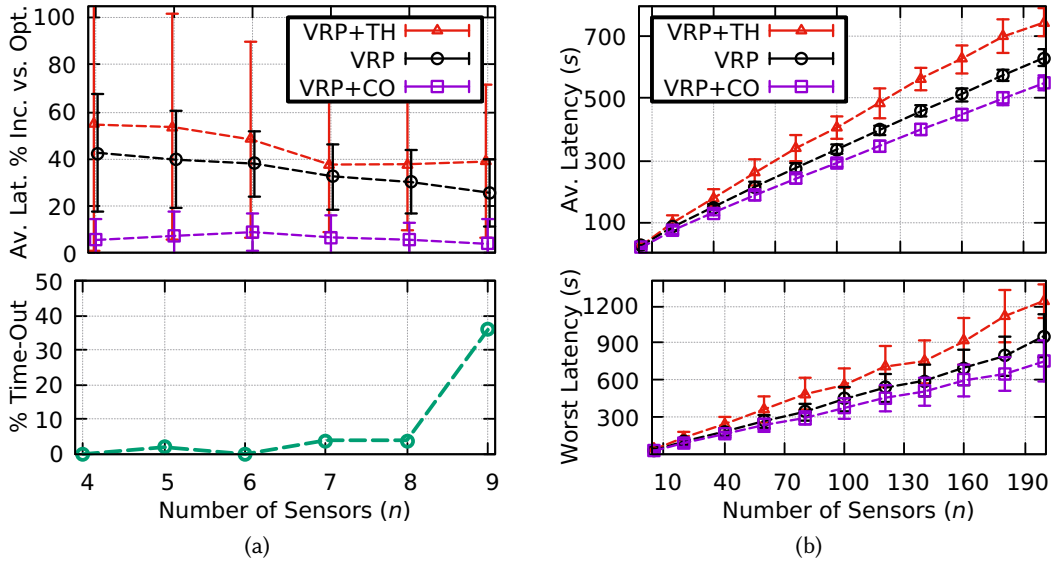



Fig. 8. Simulation results showing how the number of sensors impacts performance. Plot (a) shows the percent increase in average latency compared to the best-found solution (top) and the percentage of inputs where the MINLP solver timed out (bottom). Plot (a) shows the average latency (top) and worst latency (bottom) on larger sensor inputs.

*6.3.1 Small Varying Number of Sensors.* We ran the MINLP and the heuristics-based approaches on the *n4-30* set with two drones (the drones from [40] and [9]). To prevent excessive computation times, we set the MINLP to

timeout after 4 hours. We stopped the MINLP approach at nine sensors because the solver began to hit the timeout too frequently. The top graph of Fig. 8a shows the percent increase in average latency of the heuristics-based algorithms when compared against the optimal solution found using the MINLP and the bottom graph shows how frequently the MINLP hit the timeout. The *VRP+CO* algorithm averages 5.4% of the optimal solution while the *VRP* and *VRP+TH* averaged within 24.6% and 27.0% of the optimal, respectively. The MINLP is reasonably efficient until nine sensors, when it timed out on 36% of the inputs.

*6.3.2 Large Varying Number of Sensors.* To determine how well each of the heuristics-based algorithms can handle larger input sizes, we ran all three algorithms on the *n5-200* set with all three drones available. Figure 8b shows the average found average latency (top graph) and the worst latency (bottom graph) using each algorithm. On average, the *VRP+CO* algorithm improves the *VRP* approach by 13.3% and the *VRP+TH* algorithm by 27.7%.

*6.3.3 Varying Number of Drones.* To evaluate the impact of the number of drones on the performance of each algorithm, we fixed the number of sensors at 75 and increased the number of drones from one up to 10. The problem inputs come from the *n5-200* set. Figure 9a shows the average found average latency (top graph) and the worst latency (bottom graph) for each algorithm. These results show diminishing returns for both metrics as the number of drones increases. Despite this fact, the *VRP+CO* algorithm improves average latency compared to the *VRP+TH* and *VRP* algorithms by 26.2% and 11.5%, respectively, nearly uniformly as the number of drones increases. Comparable performance is seen for the worst latency metric.

*6.3.4 Varying Data Volume.* To investigate how optimizing the *travel-to-TX* tradeoff impacts performance, we ran each of the heuristics-based algorithms on the *q0-16* input set with three drones. Figure 9b shows the average found average latency (top graph) and the worst latency (bottom graph) using each algorithm. The *VRP+TH* algorithm initially performs well when the volume of data is at or below 1 MB. However, this method begins to

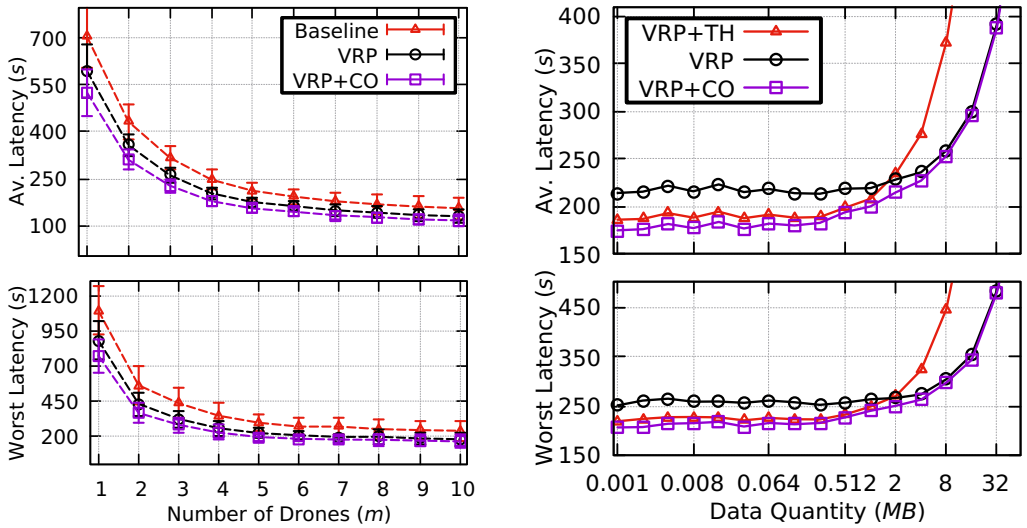



Fig. 9. Simulation results showing the average latency (top) and worst latency (bottom) while increasing the number of drones (a) and increasing the quantity of data to collect from each sensor (b).

suffer greatly when the data volume surpasses 2 MBs. This is because this method uses a simplified model for communication and is agnostic to the *travel-to-TX* relationship. The *VRP* is also agnostic to data transfer rate and data volume, taking instead the more conservative approach and always minimizing the drone-to-sensor distance. This behavior causes the algorithm to not perform as well as the other two when the data volume is below 2 MBs. However, at around 8 MBs this approach merges with our *VRP+CO* algorithm, at which point it pays off to minimize the drone-to-sensor distance to maximize the data transfer rate. Our proposed *VRP+CO* algorithm consistently outperforms the other two approaches at both small and large data volumes.

*6.3.5 Computation Time.* Table 3 shows the computation time for the various heuristics-based algorithms as the number of sensors (top) and drones (bottom) increase. We fixed the number of drones at three when increasing the number of sensors and fixed the number of sensors at 75 when increasing the number of drones. On smaller inputs (10 sensors) we see almost twice the computation to run the *VRP* algorithm when compared to the other two algorithms (*VRP+TH* and *VRP*). However, the computation time remains under 100 milliseconds until around 40 sensors. At larger inputs (200 sensors) we no longer see a notable difference between the *VRP+CO* and the *VRP+TH* algorithms, with the simpler *VRP* algorithm only slightly outperforming the others. Furthermore, computation times grow proportionally with the number of drones. The *VRP* and *VRP+TH* algorithms maintain a slight computational advantage over the *VRP+CO* approach as the number of drones increase. This computational advantage is consistent with what is seen around $n = 50$ on the top graph, showing us that the number of sensors has a larger impact on computational performance across the board when compared to the number of drones.

Table 3. Average computation time (in seconds) as the number of sensors (top) and drones (bottom) increases.

| $n =$ | 10 | 20 | 30 | 40 | 50 | 100 | 150 | 200 |
|---|---|---|---|---|---|---|---|---|
| *VRP+TH* | 0.04 | 0.05 | 0.06 | 0.07 | 0.08 | 0.34 | 0.82 | 1.83 |
| *VRP* | 0.05 | 0.06 | 0.07 | 0.07 | 0.09 | 0.34 | 0.79 | 1.70 |
| *VRP+CO* | 0.09 | 0.09 | 0.10 | 0.11 | 0.13 | 0.41 | 0.91 | 1.81 |
| *OAP* | 0.11 | 0.22 | 0.33 | 0.45 | 0.58 | - | - | - |

| $m =$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| *VRP+TH* | 0.15 | 0.15 | 0.17 | 0.21 | 0.25 | 0.29 | 0.33 | 0.36 | 0.39 | 0.40 |
| *VRP* | 0.15 | 0.15 | 0.18 | 0.21 | 0.26 | 0.31 | 0.34 | 0.36 | 0.37 | 0.44 |
| *VRP+CO* | 0.19 | 0.19 | 0.23 | 0.26 | 0.29 | 0.33 | 0.37 | 0.40 | 0.44 | 0.47 |

*6.3.6 Offline Global Planner Evaluation Summary.* These results show that our *VRP+CO* algorithm performs very near the optimal solution and is able to exploit the *travel-to-TX* tradeoff. We believe that the latter trait is the main reason that the algorithm finds better solution quality when compared against the *VRP* and the *VRP+TH* algorithms. Although the *VRP* and the *VRP+TH* algorithms are faster than the *VRP+CO* algorithm at smaller input sizes, the *VRP+CO* algorithm can still solve inputs within a reasonable time (under 100 milliseconds for less than 40 sensors) and matches the computation time of the other two on larger inputs.

## 6.4 Online Adaptive Planner Evaluation

To evaluate our proposed Online Adaptive Planner (*OAP*), we ran the algorithm multiple times on the RPi4 mounted on our drone and recorded how quickly it could solve various input sizes. We created randomized problem setups by selecting sensors from the *n5-200* input set and then running our online planner to find an adapted plan. We varied the number of sensors from 10 up to 50.

Table 3 shows the computation time required to run the online planner. The planner can handle inputs of 10 sensors or less in about 100 milliseconds and appears to scale nearly linearly with the number of sensors, taking 580 milliseconds for 50 sensors. Although this time is reasonably fast, in practice this computation is done while the drone hovers. If online computation time becomes a concern, then we recommend cutting down the considered problem. For example, only planning for the next 10 sensors rather than 50 sensors and using the original plan from the Offline Global Planner to service the latter 40 sensors.

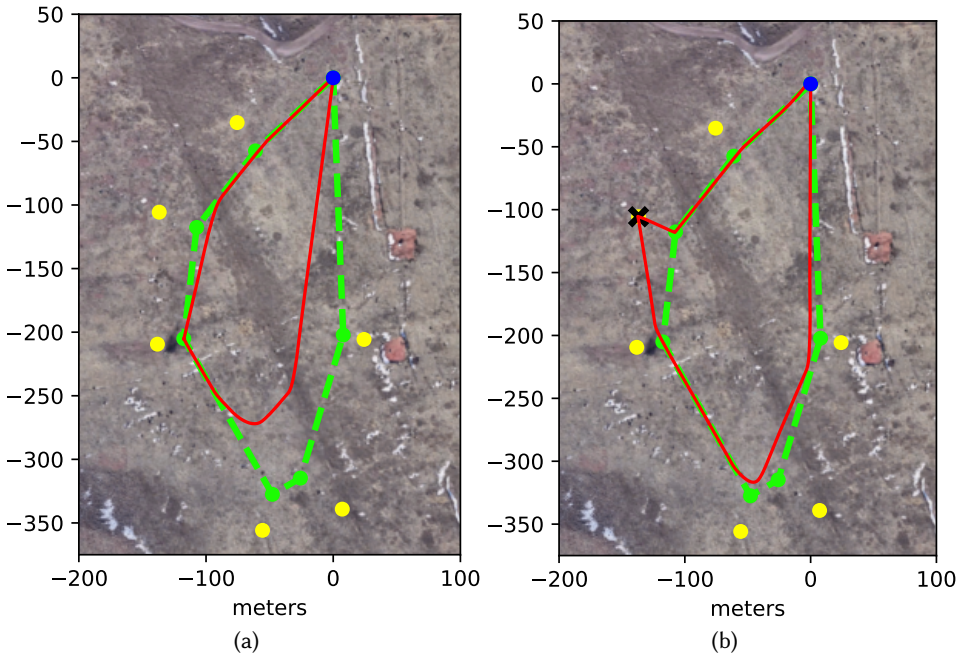



Fig. 10. Field experiment results. Blue marker is the data center, yellow markers are IoT devices, black 'X' shows a dead sensor, green lines are the Offline Global Planner's plan, and red lines are the drone's GPS trace. Panel (*a*) shows standard execution and panel (*b*) shows execution with dead sensor.

## 7 Field Prototype

To validate our data-driven methodology, we prototyped the problem setup on a physical testbed. We deployed six Raspberry Pis (three RPi3s and three RPi4s) in a large field and used our planning framework to find a consistent deployment plan. We then flew a ROARQuad drone [18] through the field, guided by our autopilot and Online Adaptive Planner, and collected data from the sensors over WiFi. The results of our prototype are shown in Fig. 10 and Fig. 3 shows the hardware that we used.

Figure 10a shows the final plan produced by the Offline Planner (in green) and the GPS trace of the drone while executing the plan (in red). We executed the plan five times and recorded the following flight times (in seconds): 84.0, 114.0, 94.0, 88.8, 123.0. This is an average of 100.8 seconds. Interestingly, the drone was able to collect data from the sensors before reaching most of the planned waypoints and did not actually stop at most of them. We ran the same setup in our Drone and Networking Simulator and recorded the following times (in seconds): 120.1, 107.5, 108.4, 108.9, 113.4. This gives us an average of 111.7 seconds in the simulator. The Offline

Global Planner predicted that it would take the drone 108.4 seconds. This gives us an error of 10.8% between the simulation and the physical hardware, an error of 3.0% between the offline planner and the simulator, and an error of 7.5% between the offline planner and the hardware. We speculate that the difference between the simulator and our field test comes from the acceleration. Our drone can accelerate very aggressively but the drone simulator does not readily support setting a custom acceleration. Other factors that may be contributing to these differences include wind and air pressure, which are not modeled in our simulation setup nor in our offline planner. Although not exact, these numbers are reasonably close, and much more accurate than the results in [8], where they had over 45% error between their planner and the physical testbed.

We repeated our experiment with one of the sensors turned off to prototype how the Online Adaptive Planner reacts to a dead sensor. Figure 10b shows the results of this experiment. We see that the online planner successfully redirected the drone to fly over the dead sensor before continuing on to the next sensor.

## 8 Limitations

While our simulation experiments and field prototype empirically demonstrate the utility of our proposed framework, there are several limitations to this current body of work that should be acknowledged.

Our empirical communication model and travel-to-TX tradeoff parameters were derived using 2.4 GHz 802.11n Wi-Fi on Raspberry Pi hardware in ad-hoc mode. While the *methodology* of profiling hardware to calibrate the Monte Carlo simulator is hardware-agnostic, the specific parameters, and potentially the structural fit of the Weibull distribution, are tied to this hardware. Furthermore, while the empirical model presented in Equ. (1) provided a robust fit for our specific testbed, deployments utilizing notably different radio technologies (e.g., LoRa, BLE, 5G) will require re-profiling both the data transfer rate model and the tradeoff parameters. Additionally, moving to heterogeneous radio configurations would require the user to model each unique pair-wise combination of radio types.

The physical tests were conducted in open-field environments and are strictly a proof-of-concept for a single drone scenario with sparsely deployed sensors. Consequently, our fitted models account for free-space path loss and basic ground reflection, but do not capture severe multi-path fading, signal shadowing from dense foliage, or urban canyon effects (Non-Line-of-Sight). In highly obstructed environments, the variance in data transmission rates would likely increase. While our framework is designed to handle such stochasticity, the calibration loop would be forced to learn a much more conservative energy margin ($\alpha$), which could significantly reduce the total number of sensors a single drone can visit.

The simulated and physical evaluations were conducted at a modest scale (up to 30 sensors). At this scale, our assumption that MAC-layer packet collisions are negligible holds reasonably well. However, scaling to hundreds of densely packed sensors could introduce severe interference not fully captured by our current network simulator, necessitating a higher-fidelity simulator (e.g., NS3) in the feedback loop. We expect that added noise from MAC-layer packet collisions would prolong hovering time and, if not accurately modeled in the Monte Carlo simulations, would lead to higher energy failure rates during deployment. Another consideration for scaling real-world deployments is the number of drones. If there were a large number of drones operating close to one another, relative to the communication range of the wireless medium being used, then we should expect packet collision issues between drones.

To evaluate our framework, we developed SimpleNetSim, an empirically driven networking simulator that intentionally abstracts the communication stack using an application-layer throughput model derived directly from our field measurements. While this simulator makes it easy to quickly model application-layer throughput on specific hardware for IoT deployments in sparse environments, it does not faithfully represent complex IoT networking scenarios, such as congested wireless channels or Non-Line-of-Sight conditions, that high-fidelity simulators like NS3 can capture. Notwithstanding, our proposed framework and planning methodology is network

simulator-agnostic, and NS3 could be integrated into the framework if configured correctly for the wireless environment under test.

While specific parameters are subject to the wide range of factors that impact wireless communication, we argue that the gains observed in our empirical evaluations are fundamentally rooted in the stochastic, distance-dependent characteristics of RF communication. Therefore, we expect the architectural benefits of our holistic framework to persist when adapting the system to new hardware and environments.

Another notable assumption that should be acknowledged is sensor location. Our formulation assumes that sensor locations are static and known *a priori*, but we should expect minor positional uncertainties due to GPS drift or physical disturbances (e.g., wind). Our framework handles these minor location uncertainties implicitly; if a node's physical shift causes a drop in the expected data transfer rate, the Online Adaptive Planner compensates by dynamically adjusting the drone's hovering time or waypoint positioning to fulfill the data requirement and these deployment statistics are then incorporated for future data collection evolutions. However, scenarios involving mobile sensors are outside the scope of our current offline-online methodology and would require continuous trajectory tracking or dynamic rendezvous node estimation.

## 9 Conclusions

In this work we presented a holistic drone planning framework for solving the Multi-Drone Data-Muling problem. Our framework utilizes an Offline Global Planner to determine how to divide sensors up between drones and what route each drone should take to collect data from its assigned sensors. The framework then validates these offline plans through Monte Carlo simulations in a data-driven drone and network simulator that is designed to replicate real-world networking performance with an Online Adaptive Planner that alters behavior during deployment in response to communication failures. Using feedback from the simulations, the framework makes input parameter adjustments to the offline planner to prevent the drones from running out of energy during deployment. Our simulation experiments show that our proposed framework finds a balance between planning to utilize all of a drone's battery in the offline planner and saving energy for adaptive behavior during deployment.

Furthermore, we proposed two Offline Global Planners and an Online Adaptive Planner that exploit a tradeoff between the time required to collect data from a sensor and the time to move closer to a sensor to increase data transmission rates. Our results show that this tradeoff is important and can greatly improve solution performance.

Future work could further investigate other limitations, such as limited drone data storage or deadlines for data delivery. Other metrics could also be explored, such as prioritizing certain sensors over others, limiting the number of allowed battery swaps or considering dynamic IoT nodes. Additionally, we see a need to consider how to adjust the deployment strategy of the entire fleet of drones during deployment when handling stochastic communication links – a feature currently missing in our proposed Online Adaptive Planner.

## Acknowledgments

We would like to thank the students in the Pervasive Computing Systems lab at Colorado School of Mines, notably Cody Fellinge, Matthew Hatch, and Jack "Siggy" Sigler, for their help towards our field experiments.

## References

[1] Safwan Alfattani, Wael Jaafar, Halim Yanikomeroglu, and Abbas Yongacoglu. 2019. Multi-UAV data collection framework for wireless sensor networks. In *Global Communications Conference (GLOBECOM)*. IEEE.

[2] Asad Arfeen, Krzysztof Pawlikowski, Don McNickle, and Andreas Willig. 2019. The role of the Weibull distribution in modelling traffic in Internet access and backbone core networks. *Journal of Network and Computer Applications* 141 (2019). doi:10.1016/j.jnca.2019.05.002

[3] Harald Bayerlein, Mirco Theile, Marco Caccamo, and David Gesbert. 2021. Multi-UAV path planning for wireless data harvesting with deep reinforcement learning. *IEEE Open Journal of the Communications Society* 2 (2021).

[4] Ilham Benmad, Elmahdi Driouch, and Mustapha Kardouchi. 2022. Trajectory Planning for Data Collection in Multi-UAV Assisted WSNs. In *Vehicular Technology Conference: (VTC2022-Spring)*. doi:10.1109/VTC2022-Spring54318.2022.9861010

[5] Francesco Betti Sorbelli, Alfredo Navarra, Lorenzo Palazzetti, Cristina M. Pinotti, and Giuseppe Prencipe. 2022. Optimal and Heuristic Algorithms for Data Collection by Using an Energy- and Storage-Constrained Drone. In *Algorithmics of Wireless Networks*, Thomas Erlebach and Michael Segal (Eds.). Springer International Publishing, Cham.

[6] Qing Cai, Zheng Tang, and Chuan Liu. 2024. Joint trajectory, transmission time and power optimization for multi-UAV data collecting system. *Heliyon* 10, 5 (2024). doi:10.1016/j.heliyon.2024.e26627

[7] Mengyu Chen, Weifa Liang, and Sajal K Das. 2021. Data collection utility maximization in wireless sensor networks via efficient determination of UAV hovering locations. In *International Conference on Pervasive Computing and Communications (PerCom)*. IEEE.

[8] Jonathan Diller, Peter Hall, and Qi Han. 2023. Holistic path planning for multi-drone data collection. In *International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*. IEEE.

[9] Jonathan Diller, Qi Han, Robert Byers, James Dotterweich, and James Humann. 2025. Hitchhiker's Guide to Patrolling: Path-Finding for Energy-Sharing Drone-UGV Teams. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.

[10] Jonathan Diller, Jack Sigler, and Qi Han. 2026. Collaborative Path Planning of Energy-Sharing Drone-UGV Teams for Persistent Patrolling. *ACM J. Auton. Transport. Syst.* 3, 4, Article 21 (July 2026), 27 pages. doi:10.1145/3788877

[11] Eslam Eldeeb, Mohammad Shehab, and Hirley Alves. 2023. Age Minimization in Massive IoT via UAV Swarm: A Multi-agent Reinforcement Learning Approach. In *Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC)*. IEEE. doi:10.1109/PIMRC56721.2023.10293964

[12] Federal Aviation Administration. 2021. *Small Unmanned Aircraft System (Small UAS)*. Advisory Circular AC 107-2A. U.S. Department of Transportation, Federal Aviation Administration, Washington, DC.

[13] Martin A Fischler and Robert C Bolles. 1981. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM* 24, 6 (1981).

[14] Xingxia Gao, Xiumin Zhu, and Linbo Zhai. 2023. AoI-Sensitive Data Collection in Multi-UAV-Assisted Wireless Sensor Networks. *IEEE Transactions on Wireless Communications* 22, 8 (2023). doi:10.1109/TWC.2022.3232366

[15] Oussama Ghdiri, Wael Jaafar, Safwan Alfattani, Jihene Ben Abderrazak, and Halim Yanikomeroglu. 2020. Energy-Efficient Multi-UAV Data Collection for IoT Networks with Time Deadlines. In *Global Communications Conference (GLOBECOM)*. IEEE, 1–6. doi:10.1109/GLOBECOM42002.2020.9322626

[16] Jie Gong, Tsung-Hui Chang, Chao Shen, and Xiang Chen. 2018. Flight Time Minimization of UAV for Data Collection Over Wireless Sensor Networks. *IEEE Journal on Selected Areas in Communications* 36, 9 (2018). doi:10.1109/JSAC.2018.2864420

[17] Peter Hall, Jonathan Diller, Ava Moon, and Qi Han. 2023. DroNS-3: Framework for Realistic Drone and Networking Simulators. In *Workshop on Micro Aerial Vehicle Networks, Systems, and Applications* (Helsinki, Finland) *(DroNet '23)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3597060.3597239

[18] Matthew Hatch, Cody Fellinge, Jonathan Diller, and Qi Han. 2025. Demo: ROARQuad: Robust, Open Academic Research Quadcopter. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services* (Hilton Anaheim, Anaheim, CA, USA) *(MobiSys '25)*. Association for Computing Machinery, New York, NY, USA, 633–634. doi:10.1145/3711875.3734378

[19] Keld Helsgaun. 2000. An effective implementation of the Lin–Kernighan traveling salesman heuristic. *European journal of operational research* 126, 1 (2000).

[20] Keld Helsgaun. 2017. An extension of the Lin-Kernighan-Helsgaun TSP solver for constrained traveling salesman and vehicle routing problems. *Roskilde* (2017).

[21] ITU-R. 2021. *Attenuation in vegetation.* Recommendation P.833-10. International Telecommunication Union. https://www.itu.int/dms_pubrec/itu-r/rec/p/R-REC-P.833-10-202109-I!!PDF-E.pdf

[22] Pejman A Karegar and Adnan Al-Anbuky. 2021. Travel path planning for UAV as a data collector for a sparse WSN. In *International conference on distributed computing in sensor systems (DCOSS)*. IEEE.

[23] Donghyun Kim, Baraki H Abay, RN Uma, Weili Wu, Wei Wang, and Alade O Tokuta. 2012. Minimizing data collection latency in wireless sensor network with multiple mobile elements. In *International Conference on Computer Communications (INFOCOM)*. IEEE.

[24] Lin Li, Lei Wang, Jiawang Hou, Junjie Ma, and Yang Liu. 2024. Multi-UAV Path Planning Based on DRL for Data Collection in UAV-Assisted IoT. In *International Conference on Information Science and Technology (ICIST)*. doi:10.1109/ICIST63249.2024.10805321

[25] Min Li, Xiao Liu, and Hong Wang. 2023. Completion Time Minimization Considering GNs' Energy for UAV-Assisted Data Collection. *IEEE Wireless Communications Letters* 12, 12 (2023). doi:10.1109/LWC.2023.3309625

[26] Yuchen Li, Weifa Liang, Wenzheng Xu, Zichuan Xu, Xiaohua Jia, Yinlong Xu, and Haibin Kan. 2021. Data collection maximization in IoT-sensor networks via an energy-constrained UAV. *Transactions on Mobile Computing* 22, 1 (2021).

[27] Xuan Lin. 2026. Data-driven acceleration of mixed-integer bilinear programs: a comparative study for robot motion planning. *Frontiers in Robotics and AI* Volume 12 - 2025 (2026). doi:10.3389/frobt.2025.1656564

[28] Tianyu Liu, Guangchi Zhang, Miao Cui, Changsheng You, Qingqing Wu, Shaodan Ma, and Wei Chen. 2023. Task Completion Time Minimization for UAV-Enabled Data Collection in Rician Fading Channels. *IEEE Internet of Things Journal* 10, 2 (2023). doi:10.1109/JIOT.2022.3204658

[29] Zhilong Liu, Raja Sengupta, and Alex Kurzhanskiy. 2017. A power consumption model for multi-rotor small unmanned aircraft systems. In *International Conference on Unmanned Aircraft Systems (ICUAS)*. doi:10.1109/ICUAS.2017.7991310

[30] Alan Mainwaring, David Culler, Joseph Polastre, Robert Szewczyk, and John Anderson. 2002. Wireless sensor networks for habitat monitoring. In *international workshop on Wireless sensor networks and applications*. ACM.

[31] Chixiong Mao, Juan Liu, and Lingfu Xie. 2020. Multi-UAV aided data collection for age minimization in wireless sensor networks. In *International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE.

[32] Rezoan Ahmed Nazib and Sangman Moh. 2021. Energy-efficient and fast data collection in UAV-aided wireless sensor networks for hilly terrains. *IEEE Access* 9 (2021).

[33] Ivo Nowak. 2015. *Column Generation based Alternating Direction Methods for solving MINLPs*. Preprint. HAW Hamburg, Department of Mechanical Engineering, Hamburg, Germany. https://optimization-online.org/?p=13751 Optimization Online e-print.

[34] Nawaf Qasem Hamood Othman, Haithm M. Al-Gunid, and Imran Younas. 2024. Efficient Multi-UAV Path Planning for Wireless Data Harvesting: A Deep Reinforcement Learning Approach. In *International Conference on Electronic Information and Communication Technology (ICEICT)*. doi:10.1109/ICEICT61637.2024.10670944

[35] Yuwen Pan, Yuanwang Yang, and Wenzao Li. 2021. A deep learning trained by genetic algorithm to improve the efficiency of path planning for data collection with multi-UAV. *IEEE Access* 9 (2021).

[36] Florian A Potra and Stephen J Wright. 2000. Interior-point methods. *Journal of computational and applied mathematics* 124, 1-2 (2000).

[37] Zhen Qin, Aijing Li, Chao Dong, Haipeng Dai, and Zhengqin Xu. 2019. Completion time minimization for multi-UAV information collection via trajectory planning. *Sensors* 19, 18 (2019).

[38] Mehmet Safak. 2017. *Digital communications*. John Wiley & Sons.

[39] Vihar Shah, Manish Kumar, and Abhishek Jindal. 2024. An Energy Efficient UAV Path Planning for Data Collection in Multi-Node IoT System. In *International Conference on Advanced Networks and Telecommunications Systems (ANTS)*. IEEE, 1–6. doi:10.1109/ANTS63515.2024.10898404

[40] Feng Shan, Junzhou Luo, Runqun Xiong, Wenjia Wu, and Jiashuo Li. 2020. Looking before Crossing: An Optimal Algorithm to Minimize UAV Energy by Speed Scheduling with a Practical Flight Energy Model. In *IEEE Conference on Computer Communications*. doi:10.1109/INFOCOM41043.2020.9155376

[41] Manish Kumar Singh, Amit Choudhary, Sandeep Gulia, and Anurag Verma. 2022. Multi-objective NSGA-II optimization framework for UAV path planning in an UAV-assisted WSN. *J. Supercomput.* 79, 1 (July 2022). doi:10.1007/s11227-022-04701-2

[42] Francesco Betti Sorbelli, Alfredo Navarra, Lorenzo Palazzetti, Cristina M Pinotti, and Giuseppe Prencipe. 2024. Wireless IoT sensors data collection reward maximization by leveraging multiple energy- and storage-constrained UAVs. *J. Comput. System Sci.* 139 (2024). doi:10.1016/j.jcss.2023.103475

[43] International Telecommunication Union. 2015. Effects of building materials and structures on radiowave propagation above about 100 MHz.

[44] Kehao Wang, Zheng Tang, Pei Liu, Yirui Cong, Xiangke Wang, Dejin Kong, and Yitong Li. 2021. UAV-Based and Energy-Constrained Data Collection System with Trajectory, Time, and Collection Scheduling Optimization. In *International Conference on Communications in China (ICCC)*. IEEE/CIC. doi:10.1109/ICCC52777.2021.9580291

[45] Hui Xiao, Zhigang Hu, Kun Yang, Yao Du, and Dongwei Chen. 2020. An Energy-Aware Joint Routing and Task Allocation Algorithm in MEC Systems Assisted by Multiple UAVs. In *International Wireless Communications and Mobile Computing (IWCMC)*. doi:10.1109/IWCMC48107.2020.9148519

[46] Yong Zeng, Jie Xu, and Rui Zhang. 2019. Energy minimization for wireless communication with rotary-wing UAV. *Transactions on Wireless Communications* 18, 4 (2019).

[47] Peng Zhang, Baoyu Liu, Sihua Gao, Bingyang Zhao, and Weifeng Xu. 2024. A Multi-UAV Trajectory Planning Algorithm for Time-Sensitive Data Collection. In *International Conference on Next Generation Data-driven Networks (NGDN)*. doi:10.1109/NGDN61651.2024.10744152

[48] Rongtao Zhang, Jie Hao, Ran Wang, Hui Wang, Hai Deng, and Shifang Lu. 2023. Global Path Planning for Multi-objective UAV-Assisted Sensor Data Collection: A DRL Approach. In *Web and Big Data. APWeb-WAIM 2022 International Workshops*, Shiyu Yang and Saiful Islam (Eds.). Springer Nature Singapore, Singapore.

[49] Le Zhao, Zesong Fei, Jingxuan Huang, Xinyi Wang, Bin Li, and Weijie Yuan. 2026. Deployment Design for Multi-UAV-Assisted IoT Networks: A Digital Twin-Driven Deep Reinforcement Learning Approach. *IEEE Transactions on Wireless Communications* 25 (2026), 2468–2483. doi:10.1109/TWC.2025.3596864

[50] Conghao Zhou, Hongli He, Peng Yang, Feng Lyu, Wen Wu, Nan Cheng, and Xuemin Shen. 2019. Deep RL-based trajectory planning for AoI minimization in UAV-assisted IoT. In *International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE.

[51] Zheng Zhou, Juan Liu, and Chixiong Mao. 2024. Age of Information Oriented Data Collection via Energy-Constrained UAVs in Wireless Sensor Networks. *IEEE Access* 12 (2024). doi:10.1109/ACCESS.2024.3356175

[52] Botao Zhu, Ebrahim Bedeer, Ha H Nguyen, Robert Barton, and Zhen Gao. 2022. UAV Trajectory Planning for AoI-Minimal Data Collection in UAV-Aided IoT Networks by Transformer. *IEEE Transactions on Wireless Communications* (2022).

[53] Botao Zhu, Ebrahim Bedeer, Ha H Nguyen, Robert Barton, and Jerome Henry. 2021. UAV trajectory planning in wireless sensor networks for energy consumption minimization by deep reinforcement learning. *IEEE Transactions on Vehicular Technology* 70, 9 (2021).

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009